

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO KỸ THUẬT ASSIGNMENT 07
MÔN HỌC: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn : PGS.TS. Trần Đình Quế
Lớp : D23CTPM01
Nhóm học phần : CT01
Thành viên thực hiện : Nguyễn Nam Hải
Mã sinh viên : B23DCCN277

Hà Nội – 2026

Mục lục

1	Giới thiệu chung về Bài tập lớn	4
2	Mục tiêu và Phạm vi	5
2.1	Mục tiêu của báo cáo	5
2.2	Phạm vi thực hiện	5
3	Cấu trúc của báo cáo	7
4	Lịch sử phát triển của Trí tuệ nhân tạo	9
4.1	Sự ra đời của Trí tuệ nhân tạo (Trước 1956 và Hội nghị Dartmouth)	10
4.2	Kỳ vọng ban đầu và Mùa đông AI đầu tiên (1956 - 1980)	11
4.3	Hệ chuyên gia và Mùa đông AI thứ hai (1980 - 1993)	12
4.4	Sự trỗi dậy của Thuyết kết nối và Học máy (1990s - 2010s)	14
4.5	Sự bùng nổ của Học sâu (2011 - nay)	16
4.6	Tình trạng hiện tại và Tương lai của Trí tuệ nhân tạo	18
5	Các kỹ thuật Học máy Cơ bản (Machine Learning)	20
5.1	Giới thiệu chung về các kỹ thuật Học máy cơ bản	20
5.2	Bộ dữ liệu 1: Diabetes (Bệnh Tiểu đường)	21
5.2.1	Mô tả bộ dữ liệu và phân tích	21
5.2.2	Lý thuyết Hồi quy Logistic (Logistic Regression)	21
5.2.3	Cài đặt mô hình	23
5.3	Bộ dữ liệu 2: Housing Prices Vietnam (Giá Nhà tại Việt Nam)	25
5.3.1	Mô tả bộ dữ liệu và phân tích	25
5.3.2	Lý thuyết Hồi quy Tuyến tính Đa biến (Multiple Linear Regression)	25
5.3.3	Cài đặt mô hình	27
5.4	Bộ dữ liệu 3: Ecommerce Reviews (Đánh giá Thương mại Điện tử)	29
5.4.1	Mô tả bộ dữ liệu và phân tích	29
5.4.2	Lý thuyết Mạng nơ-ron đa tầng (Multilayer Perceptron - MLP)	29
5.4.3	Cài đặt mô hình	31
5.5	Đánh giá và So sánh các Mô hình	33
6	Mạng Nơ-ron Tích chập (CNN)	35
6.1	Cơ sở Lý thuyết của Mạng Nơ-ron Tích chập	36
6.1.1	Phép toán Tích chập (Convolution Operation)	36
6.1.2	Padding và Strides	36
6.1.3	Hàm Kích hoạt Phi tuyến (Non-linear Activation)	37
6.1.4	Lớp Gộp (Pooling Layer)	37
6.2	Tập dữ liệu 1: CIFAR-10	38
6.2.1	Mô tả Dữ liệu	38
6.2.2	Cài đặt Tích chập - Từ cơ bản (NumPy)	38
6.2.3	Triển khai trên Keras (CIFAR-10)	39
6.2.4	Triển khai trên PyTorch (CIFAR-10)	40
6.3	Tập dữ liệu 2: MNIST	42
6.3.1	Mô tả Dữ liệu	42
6.3.2	Cài đặt Tích chập - Từ cơ bản (NumPy)	42

6.3.3	Triển khai trên Keras (MNIST)	42
6.3.4	Triển khai trên PyTorch (MNIST)	43
6.4	Tập dữ liệu 3: Diabetes 1D	44
6.4.1	Mô tả Dữ liệu	44
6.4.2	Cài đặt Tích chập 1 Chiều - Từ cơ bản (NumPy)	44
6.4.3	Triển khai trên Keras (Diabetes 1D)	45
6.4.4	Triển khai trên PyTorch (Diabetes 1D)	45
6.5	Đánh giá và So sánh	47
7	Mạng Nơ-ron Hồi quy (RNN) và Xử lý Dữ liệu Chuỗi Thời gian	48
7.1	Cơ sở Lý thuyết cơ bản	48
7.1.1	Mạng Nơ-ron Hồi quy (RNN)	48
7.1.2	Lan truyền ngược qua thời gian (BPTT)	48
7.1.3	Bộ nhớ Ngắn-Dài hạn (LSTM)	49
7.1.4	Gated Recurrent Unit (GRU)	49
7.2	Tập dữ liệu 1: Dự báo Giá Cổ phiếu Amazon (AMZN)	50
7.2.1	Mô tả, Phân phối và Phân tích Dữ liệu	50
7.2.2	Cài đặt mô hình bằng Python (From Scratch - NumPy)	50
7.2.3	Cài đặt mô hình bằng Keras	52
7.2.4	Cài đặt mô hình bằng PyTorch	52
7.3	Tập dữ liệu 2: Dự báo Giá Vàng (Gold Price)	54
7.3.1	Mô tả, Phân phối và Phân tích Dữ liệu	54
7.3.2	Cài đặt mô hình bằng Python (From Scratch - NumPy)	54
7.3.3	Cài đặt mô hình bằng Keras	55
7.3.4	Cài đặt mô hình bằng PyTorch	56
7.4	Tập dữ liệu 3: Dự báo Giá Bạc (Silver Price)	57
7.4.1	Mô tả, Phân phối và Phân tích Dữ liệu	57
7.4.2	Cài đặt mô hình bằng Python (From Scratch - NumPy)	57
7.4.3	Cài đặt mô hình bằng Keras	58
7.4.4	Cài đặt mô hình bằng PyTorch	58
7.5	Đánh giá và So sánh Mô hình	60
7.5.1	Phân tích kết quả	60
7.5.2	Kết luận về việc lựa chọn Thư viện	60
8	Triển khai Hệ thống (Deployment)	61
8.1	Tổng quan Kiến trúc Triển khai	61
8.2	Xây dựng Backend với FastAPI	61
8.3	Xây dựng Giao diện Người dùng (Web UI)	63
8.4	Xử lý Sự cố TensorFlow và Metal trên macOS	65

LỜI MỞ ĐẦU

Trong kỷ nguyên công nghệ số hiện nay, Trí tuệ Nhân tạo (AI - Artificial Intelligence) và Học máy (Machine Learning) không chỉ dừng lại ở các nghiên cứu mang tính lý thuyết trong các phòng thí nghiệm, mà đã và đang trở thành cốt lõi của hàng loạt ứng dụng thực tiễn làm thay đổi sâu sắc mọi mặt của đời sống xã hội. Từ các hệ thống gợi ý sản phẩm thông minh của các gã khổng lồ thương mại điện tử, các trợ lý ảo có khả năng giao tiếp tự nhiên với con người, cho đến các phương tiện tự hành phức tạp và các phần mềm hỗ trợ chẩn đoán y khoa với độ chính xác cao, AI đang chứng minh sức mạnh to lớn trong việc giải quyết các bài toán khó khăn của nhân loại. Tuy nhiên, để một mô hình AI từ giai đoạn nghiên cứu, huấn luyện trên các tập dữ liệu tính có thể thực sự mang lại giá trị và phục vụ hàng triệu người dùng, nó cần phải vượt qua một khoảng cách rất lớn: đó chính là quá trình triển khai mô hình vào môi trường thực tế (Model Deployment).

Triển khai mô hình AI là một quá trình kỹ thuật phức tạp, đòi hỏi sự kết hợp nhuần nhuyễn giữa kiến thức chuyên sâu về Khoa học dữ liệu (Data Science), kỹ năng lập trình, và tư duy kiến trúc hệ thống, đặc biệt là các kỹ năng thuộc lĩnh vực MLOps (Machine Learning Operations). Một mô hình đạt độ chính xác 99% trong môi trường thử nghiệm (Jupyter Notebook) hoàn toàn có thể trở nên vô dụng nếu nó yêu cầu tới 10 giây để dự đoán một kết quả, hoặc đòi hỏi lượng tài nguyên máy chủ quá lớn mà hệ thống hiện tại không thể đáp ứng, hoặc không thể xử lý hàng nghìn yêu cầu đồng thời từ người dùng cuối. Do đó, việc hiểu rõ và làm chủ các kỹ thuật triển khai, tối ưu hóa mô hình, xây dựng API giao tiếp và đóng gói ứng dụng là một trong những yêu cầu bắt buộc và quan trọng nhất đối với các kỹ sư AI hiện đại.

Bài tập lớn số 07 của học phần Hệ thống thông minh (Intelligent Systems) với chủ đề "Triển khai các mô hình Trí tuệ Nhân tạo - AI Models Deployment" được thiết kế nhằm mục đích cung cấp cho sinh viên một cái nhìn toàn diện, sâu sắc và mang tính thực chiến cao về toàn bộ vòng đời của một dự án học máy sau giai đoạn huấn luyện. Thông qua việc thực hiện bài tập lớn này, chúng em không chỉ áp dụng lại những kiến thức lý thuyết đã học mà còn phải đối mặt và giải quyết các thách thức thực tế trong việc đưa các mô hình deep learning hiện đại (như CNN, RNN, Transformers) lên các nền tảng web hoặc máy chủ, đảm bảo tính sẵn sàng, khả năng mở rộng và hiệu suất hoạt động tốt nhất.

1 Giới thiệu chung về Bài tập lớn

Sự phát triển bùng nổ của các mô hình Deep Learning trong những năm gần đây đã mở ra những hướng đi mới trong việc giải quyết các bài toán phức tạp liên quan đến thị giác máy tính (Computer Vision), xử lý ngôn ngữ tự nhiên (Natural Language Processing) và dữ liệu dạng chuỗi (Time-series data). Các framework mạnh mẽ như TensorFlow, PyTorch hay Keras đã làm cho quá trình xây dựng và huấn luyện mô hình trở nên dễ dàng hơn bao giờ hết. Hàng loạt các cấu trúc mạng nơ-ron từ đơn giản như Multi-Layer Perceptron (MLP) cho đến các cấu trúc phức tạp như ResNet, YOLO, BERT, hay GPT đã được công bố rộng rãi.

Thế nhưng, vòng đời của một dự án AI không kết thúc khi quá trình huấn luyện báo cáo hàm loss hội tụ và độ chính xác đạt ngưỡng mong muốn. Ngược lại, đó mới chỉ là điểm khởi đầu cho một chặng đường đầy thử thách tiếp theo: Đưa mô hình vào phục vụ thực tế (Production). Bối cảnh của Bài tập lớn 07 được đặt trong tình hình thực tế này, khi mà các nhà tuyển dụng và ngành công nghiệp phần mềm đang khát khao những kỹ sư không chỉ biết "train model" mà còn phải biết "deploy model".

Bài tập lớn số 07 tập trung vào việc mô phỏng lại một quy trình phát triển và triển khai phần mềm AI tiêu chuẩn trong các công ty công nghệ lớn. Yêu cầu của bài tập lớn đòi hỏi sinh viên phải tiếp nhận một (hoặc nhiều) mô hình đã được huấn luyện sẵn (pre-trained models) hoặc tự tay huấn luyện các mô hình từ đầu trên các tập dữ liệu tiêu chuẩn. Sau khi có được các tệp trọng số (weights) hoặc toàn bộ mô hình (ví dụ dạng .h5, .pt, .onnx), sinh viên sẽ phải tiến hành xây dựng các dịch vụ backend API sử dụng các vi khung (micro-frameworks) phổ biến như FastAPI hoặc Flask.

Hơn thế nữa, một yếu tố quan trọng trong kiến trúc hệ thống hiện đại là tính di động (portability) và khả năng cô lập môi trường. Do đó, bài tập lớn cũng đặt ra yêu cầu phải sử dụng các công nghệ container hóa (Containerization) điển hình là Docker. Việc đóng gói mô hình, mã nguồn API, cùng với toàn bộ các thư viện phụ thuộc vào trong một Docker container không chỉ giúp tránh tình trạng "chạy được trên máy tôi nhưng lỗi trên server" mà còn tạo tiền đề vững chắc cho việc triển khai tự động lên các hệ thống điện toán đám mây (Cloud Computing) như AWS, Google Cloud, hay Azure thông qua các công cụ điều phối (Orchestration tools) như Kubernetes.

Động lực chính (Motivation) để hoàn thành tốt bài tập lớn này không chỉ là để đáp ứng yêu cầu học thuật của môn học, mà quan trọng hơn cả, là nhằm rèn luyện và trang bị những kỹ năng thiết yếu mà một AI Engineer / Machine Learning Engineer chuyên nghiệp cần có. Quá trình làm bài tập lớn giúp chúng em cọ xát với các khái niệm phức tạp như độ trễ (latency), thông lượng (throughput), tối ưu hóa bộ nhớ, và bảo mật dữ liệu – những yếu tố quyết định sự sống còn của một sản phẩm AI trên thị trường thực tế.

2 Mục tiêu và Phạm vi

2.1 Mục tiêu của báo cáo

Dự án và báo cáo "AI Models Deployment" hướng đến những mục tiêu cốt lõi sau:

- **Củng cố và áp dụng kiến thức nền tảng:** Vận dụng các kiến thức về Học máy và Deep Learning để hiểu rõ kiến trúc, cách thức hoạt động và quá trình tiền xử lý/hậu xử lý dữ liệu đầu vào/đầu ra của các mô hình AI khác nhau.
- **Xây dựng RESTful API cho các mô hình AI:** Nắm vững kỹ thuật lập trình web backend để thiết kế và phát triển các API giao tiếp, cho phép các ứng dụng khách (client-side) có thể gửi yêu cầu dự đoán (inference requests) tới mô hình một cách dễ dàng, an toàn và hiệu quả thông qua giao thức HTTP.
- **Đóng gói và quản lý môi trường bằng Docker:** Thành thạo việc viết Dockerfile, thiết lập môi trường tối ưu (giảm thiểu kích thước image, sử dụng các base image phù hợp), và sử dụng Docker Compose để quản lý các dịch vụ đa thành phần.
- **Tối ưu hóa quá trình Inference:** Nghiên cứu và thực nghiệm các phương pháp tối ưu hóa tốc độ dự đoán của mô hình, chẳng hạn như chuyển đổi định dạng mô hình (ví dụ sang ONNX, TensorRT), sử dụng batch inference, hoặc cấu hình số lượng worker phù hợp cho web server.
- **Triển khai lên môi trường Cloud (Tùy chọn/Nâng cao):** Thực hành việc đưa ứng dụng đã được đóng gói lên các nền tảng đám mây công cộng, thiết lập CI/CD pipeline để tự động hóa quá trình kiểm thử và triển khai.

2.2 Phạm vi thực hiện

Trong khuôn khổ của Bài tập lớn 07, phạm vi của dự án được giới hạn trong các khía cạnh sau để đảm bảo tính khả thi và tập trung vào chất lượng kỹ thuật:

1. **Về mặt dữ liệu và mô hình:** Dự án sẽ tập trung triển khai một số mô hình đại diện cho các bài toán phổ biến. Cụ thể có thể bao gồm mô hình phân loại hình ảnh (Image Classification) sử dụng kiến trúc CNN (Convolutional Neural Networks), hoặc bài toán phân tích cảm xúc (Sentiment Analysis) sử dụng Transformer. Các mô hình này có thể là mô hình được huấn luyện lại trên các tập dữ liệu phổ biến, hoặc sử dụng các pre-trained models.
2. **Về công nghệ backend:** Hệ thống sẽ sử dụng FastAPI – một framework Python hiện đại, hiệu năng cao, dựa trên Starlette và Pydantic. Việc lựa chọn FastAPI dựa trên ưu điểm vượt trội về tốc độ xử lý bất đồng bộ (asynchronous) và khả năng tự động sinh tài liệu API (Swagger UI).
3. **Về quy trình đóng gói và triển khai:** Dự án bắt buộc phải cung cấp hệ thống Dockerfile chuẩn hóa, cho phép build và run ứng dụng chỉ với các lệnh Docker cơ bản. Việc triển khai (deploy) có thể thực hiện trên môi trường máy chủ cục bộ (local server), hoặc các dịch vụ triển khai đám mây.

4. **Không bao gồm:** Dự án sẽ không đi sâu vào việc tối ưu hóa kiến trúc mô hình (như Neural Architecture Search) ở mức quá phức tạp, không thực hiện thu thập dữ liệu lớn từ đầu, và không đi sâu vào việc xây dựng một giao diện frontend quá phức tạp, mà chỉ tập trung ở mức độ hiển thị cơ bản hoặc sử dụng trực tiếp API docs để demo ứng dụng.

3 Cấu trúc của báo cáo

Báo cáo này được trình bày một cách hệ thống, chi tiết và logic, tuân theo quy trình phát triển phần mềm chuẩn mực. Cấu trúc của báo cáo được chia thành 5 chương chính (không kể phần Mở đầu và Kết luận), mỗi chương giải quyết một khía cạnh cụ thể trong toàn bộ quy trình xây dựng và triển khai hệ thống AI. Dưới đây là tóm tắt nội dung chính của các chương:

Chương 1: Cơ sở lý thuyết và Tổng quan công nghệ

Chương này đóng vai trò như một nền tảng kiến thức vững chắc cho các chương thực hành phía sau. Nội dung sẽ trình bày chi tiết về các khái niệm cốt lõi trong Học máy và Deep Learning, kiến trúc của các mô hình được lựa chọn để triển khai. Đồng thời, chương này cũng phân tích chuyên sâu về các công nghệ và framework được sử dụng trong dự án, bao gồm:

- Tổng quan về kiến trúc RESTful API và lợi ích của việc microservices hóa hệ thống học máy.
- Phân tích chi tiết về framework FastAPI, Pydantic và cơ chế lập trình bất đồng bộ (async/await) trong Python.
- Lý thuyết về Containerization, so sánh Docker với các công nghệ máy ảo (Virtual Machines) truyền thống, và vai trò của Docker trong quy trình MLOps.
- Các phương pháp và định dạng tối ưu hóa mô hình AI (như ONNX, TorchScript).

Chương 2: Xây dựng và Huấn luyện Mô hình AI

Chương này mô tả quá trình chuẩn bị "bộ não" cho hệ thống. Nội dung bao gồm việc thu thập, phân tích và tiền xử lý dữ liệu (Exploratory Data Analysis & Data Preprocessing). Tiếp theo, báo cáo sẽ trình bày chi tiết quá trình thiết kế cấu trúc mạng nơ-ron. Các khía cạnh quan trọng như việc lựa chọn hàm mất mát (loss function), thuật toán tối ưu (optimizer), chiến lược học (learning rate schedule), và quá trình huấn luyện (training loop) cũng như đánh giá mô hình bằng các độ đo (metrics) sẽ được phân tích kỹ lưỡng và đi kèm các ví dụ thực nghiệm.

Chương 3: Xây dựng Dịch vụ Backend API với FastAPI

Được xem là cầu nối giữa mô hình tính toán và người dùng cuối, chương này trình bày các bước lập trình cụ thể để gói gọn mô hình vào một ứng dụng web. Các nội dung chính bao gồm:

- Thiết kế sơ đồ kiến trúc API và luồng xử lý dữ liệu.
- Viết các lớp (classes) và hàm (functions) để thực hiện tiền xử lý dữ liệu từ HTTP requests sao cho tương thích với định dạng đầu vào của mô hình.
- Quản lý vòng đời của mô hình trong ứng dụng (khởi tạo mô hình khi server start) để tránh vấn đề cold start, giảm thiểu độ trễ.
- Định nghĩa các Endpoints, kiểm tra dữ liệu đầu vào bằng Pydantic và trả về kết quả dự đoán (Inference Response) theo định dạng JSON chuẩn.

Chương 4: Đóng gói Ứng dụng với Docker

Chương này đi sâu vào thực hành kỹ thuật DevOps/MLOps, mô tả chi tiết cách thức tạo ra một Docker Image hoàn chỉnh cho ứng dụng AI Backend. Báo cáo sẽ trình bày phân tích chuyên sâu về file `Dockerfile`, giải thích ý nghĩa của từng chỉ thị và chiến lược tối ưu hóa kích thước image. Ngoài ra, việc sử dụng `docker-compose` để cấu hình và chạy hệ thống nhiều thành phần cũng sẽ được trình bày rõ ràng với mã nguồn minh họa chi tiết.

Chương 5: Triển khai, Đánh giá và Tối ưu hóa Hệ thống

Chương cuối cùng của báo cáo trình bày quá trình đưa ứng dụng lên các môi trường thực tế và các thực nghiệm đo lường hiệu năng tổng thể. Nội dung bao gồm việc trình bày chi tiết các bước triển khai (deploy), các kịch bản kiểm thử API. Đặc biệt, chương này sẽ có các số liệu, đánh giá hiệu suất của hệ thống, phân tích thời gian phản hồi (Response Time) và thông lượng dưới các điều kiện vận hành khác nhau. Dựa trên các kết quả đó, chương này sẽ đề xuất các giải pháp nhằm nâng cao khả năng vận hành ổn định cho hệ thống.

4 Lịch sử phát triển của Trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence - AI) là một trong những lĩnh vực nghiên cứu năng động và phát triển nhanh nhất trong khoa học máy tính. Mặc dù thuật ngữ "Trí tuệ nhân tạo" mới chỉ chính thức được đưa ra vào giữa thế kỷ 20, nhưng ước mơ về việc tạo ra những cỗ máy có khả năng suy nghĩ và hành động như con người đã tồn tại từ thời cổ đại qua những câu chuyện thần thoại, truyền thuyết và các tác phẩm cơ khí sơ khai. Sự phát triển của AI không phải là một đường thẳng đi lên, mà là một chuỗi các giai đoạn với những bước tiến đột phá đan xen với những kỳ vọng bị phá vỡ, thường được gọi là các "Mùa đông AI". Chương này sẽ đi sâu vào lịch sử hình thành, sự phát triển qua các thời kỳ, những cột mốc quan trọng và trạng thái hiện tại của Trí tuệ nhân tạo, đồng thời xem xét chi tiết về các cơ sở toán học và thuật toán trong những thời kỳ đầu.

Sự hình thành của AI bắt nguồn từ các lĩnh vực triết học, toán học, tâm lý học, thần kinh học và khoa học máy tính. Những câu hỏi cốt lõi như "Tâm trí hoạt động như thế nào?", "Làm thế nào một hệ thống vật lý có thể tạo ra trí thông minh?" đã thôi thúc các nhà nghiên cứu tìm kiếm cách mô phỏng các quá trình nhận thức của con người bằng các thiết bị tính toán cơ học và điện tử. Trong thế kỷ 20, sự ra đời của máy tính điện tử đã cung cấp nền tảng phần cứng cần thiết để biến những lý thuyết về trí tuệ nhân tạo thành hiện thực.

Lịch sử AI có thể được chia thành nhiều giai đoạn chính: sự khởi đầu khái niệm với Alan Turing, Hội nghị Dartmouth huyền thoại đánh dấu sự ra đời của lĩnh vực học thuật, những thập kỷ của sự lạc quan tột độ (và sự sụp đổ sau đó), sự trỗi dậy của các hệ chuyên gia, sự trở lại của mạng nơ-ron nhân tạo nhờ thuật toán lan truyền ngược, và cuối cùng là kỷ nguyên bùng nổ của Học sâu (Deep Learning) và các mô hình ngôn ngữ lớn (Large Language Models) mà chúng ta đang chứng kiến ngày nay.

4.1 Sự ra đời của Trí tuệ nhân tạo (Trước 1956 và Hội nghị Dartmouth)

Thời kỳ trước năm 1956 đánh dấu những bước chuẩn bị quan trọng về mặt lý thuyết và khái niệm cho sự ra đời của AI. Một trong những cột mốc quan trọng nhất là bài báo kinh điển "Computing Machinery and Intelligence" được xuất bản năm 1950 bởi nhà toán học, logic học và mật mã học vĩ đại người Anh, Alan Turing. Trong bài báo này, Turing đã đặt ra một câu hỏi nền tảng: "Máy móc có thể suy nghĩ không?" (Can machines think?). Để tránh những tranh cãi triết học phức tạp về định nghĩa của "suy nghĩ", Turing đã đề xuất một bài kiểm tra thực nghiệm, sau này được biết đến rộng rãi với tên gọi "Phép thử Turing" (Turing Test).

Phép thử Turing được thiết kế như một "Trò chơi bắt chước" (Imitation Game), trong đó một người thẩm vấn (interrogator) sẽ giao tiếp với một con người và một máy tính thông qua văn bản (text) ở hai phòng tách biệt. Nếu người thẩm vấn không thể phân biệt được đâu là máy tính và đâu là con người sau một khoảng thời gian giao tiếp, máy tính đó được coi là đã vượt qua phép thử và có khả năng thể hiện trí thông minh tương đương con người. Bài báo của Turing không chỉ giới thiệu Phép thử Turing mà còn thảo luận về nhiều vấn đề then chốt của AI như học máy (machine learning), thuật toán di truyền (genetic algorithms) và học củng cố (reinforcement learning).

Bên cạnh Alan Turing, những nghiên cứu đầu tiên về mạng nơ-ron nhân tạo cũng được tiến hành trong giai đoạn này. Năm 1943, Warren McCulloch (nhà thần kinh học) và Walter Pitts (nhà logic học) đã xuất bản bài báo "A Logical Calculus of the Ideas Immanent in Nervous Activity", đề xuất một mô hình toán học đơn giản đầu tiên của nơ-ron sinh học. Họ chứng minh rằng các mạng bao gồm các nơ-ron nhân tạo này có thể tính toán bất kỳ hàm logic nào.

Về mặt toán học, mô hình McCulloch-Pitts có thể được biểu diễn như sau. Mỗi nơ-ron nhận các tín hiệu đầu vào nhị phân $x_i \in \{0, 1\}$. Trọng số của các kết nối, w_i , có thể là kích thích (excitatory, $w_i > 0$) hoặc ức chế (inhibitory, $w_i < 0$). Trạng thái đầu ra y của nơ-ron được xác định bởi một hàm bước (step function):

$$y = \begin{cases} 1 & \text{nếu } \sum_{i=1}^n w_i x_i \geq \theta \\ 0 & \text{nếu } \sum_{i=1}^n w_i x_i < \theta \end{cases} \quad (1)$$

trong đó θ là ngưỡng kích hoạt (threshold). Mặc dù đơn giản, mô hình này đã đặt nền móng lý thuyết quan trọng cho cấu trúc của các mạng nơ-ron hiện đại.

Năm 1956 được coi là năm sinh chính thức của lĩnh vực Trí tuệ nhân tạo với "Dự án Nghiên cứu Mùa hè về Trí tuệ nhân tạo tại Dartmouth" (Dartmouth Summer Research Project on Artificial Intelligence). Hội nghị này được tổ chức bởi John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon. John McCarthy đã chính thức đề xuất việc sử dụng thuật ngữ "Artificial Intelligence" (Trí tuệ nhân tạo) trong đề xuất của dự án. Đề xuất của hội nghị có tuyên bố đầy tham vọng: "Nghiên cứu sẽ được tiến hành dựa trên phỏng đoán rằng mọi khía cạnh của việc học tập hoặc bất kỳ đặc điểm nào khác của trí thông minh đều có thể được mô tả chính xác đến mức một cỗ máy có thể được chế tạo để mô phỏng nó".

Hội nghị Dartmouth quy tụ những bộ óc sáng giá nhất thời bấy giờ, trong đó có Allen Newell và Herbert A. Simon, những người đã trình bày chương trình "Logic Theorist" chương trình máy tính đầu tiên được thiết kế để bắt chước kỹ năng giải quyết vấn đề của con người, và đã chứng minh thành công nhiều định lý trong cuốn Principia Mathematica. Sự kiện này đã thống nhất một cộng đồng nhỏ các nhà nghiên cứu xung quanh một mục tiêu chung và khởi xướng nhiều thập kỷ nghiên cứu chuyên sâu về AI.

4.2 Kỳ vọng ban đầu và Mùa đông AI đầu tiên (1956 - 1980)

Những năm tiếp theo sau Hội nghị Dartmouth chứng kiến một sự lạc quan tột độ và những khoản đầu tư khổng lồ từ các cơ quan chính phủ, đặc biệt là DARPA (Defense Advanced Research Projects Agency) của Hoa Kỳ. Những người tiên phong trong lĩnh vực AI tin rằng máy móc có trí thông minh ngang ngửa con người sẽ xuất hiện trong vòng một vài thập kỷ.

Trong giai đoạn này, nhiều chương trình máy tính đáng kinh ngạc đã được phát triển. Ví dụ tiêu biểu là General Problem Solver (GPS) do Newell và Simon phát triển năm 1957. GPS được thiết kế để giải quyết các vấn đề đa dạng theo cách tư duy của con người, sử dụng phương pháp phân tích phương tiện-mục đích (means-ends analysis) để giảm thiểu sự khác biệt giữa trạng thái hiện tại và trạng thái đích. Mặc dù GPS giải quyết tốt các bài toán logic đơn giản, nó gặp khó khăn khi đối mặt với các vấn đề trong thế giới thực đòi hỏi kiến thức nền rộng lớn.

Một thành tựu nổi bật khác là chương trình ELIZA, do Joseph Weizenbaum tại MIT phát triển từ năm 1964 đến 1966. ELIZA là một trong những chương trình xử lý ngôn ngữ tự nhiên đầu tiên, được thiết kế để mô phỏng một nhà trị liệu tâm lý Rogerian. Mặc dù ELIZA chỉ sử dụng các kỹ thuật đối sánh mẫu (pattern matching) và thay thế văn bản đơn giản mà không thực sự "hiểu" ngữ nghĩa, nó đã tạo ra ảo giác mạnh mẽ về trí thông minh và sự đồng cảm, khiến nhiều người dùng tin rằng họ đang trò chuyện với một thực thể có trí óc.

Tuy nhiên, sự lạc quan quá mức đã dẫn đến những hứa hẹn phi thực tế. Khi những khó khăn thực sự của việc giải quyết các bài toán AI phức tạp lộ diện, tiến độ chậm lại đáng kể. Các hệ thống AI thời kỳ này chủ yếu dựa trên logic biểu tượng (symbolic logic) và các quy tắc tìm kiếm heuristic. Mặc dù phương pháp này có hiệu quả với các "thế giới nhỏ" (micro-worlds) bị giới hạn, nó hoàn toàn thất bại khi đối mặt với "sự bùng nổ tổ hợp" (combinatorial explosion) trong các vấn đề thế giới thực quy mô lớn. Số lượng các trạng thái có thể xảy ra trong các bài toán thực tế tăng lên theo cấp số nhân, vượt xa khả năng tính toán của máy tính và bộ nhớ phần cứng lúc bấy giờ.

Hơn nữa, một rào cản lớn khác là "nghịch lý Moravec" (Moravec's paradox). Nghịch lý này phát biểu rằng các nhiệm vụ tư duy cấp cao (như giải toán logic, chơi cờ) đòi hỏi rất ít tính toán, trong khi các nhiệm vụ nhận thức và vận động cấp thấp (như nhận dạng hình ảnh, di chuyển trong không gian, hiểu ngôn ngữ giao tiếp hàng ngày) lại đòi hỏi những nguồn lực tính toán khổng lồ.

Báo cáo Lighthill (Lighthill Report) năm 1973 tại Anh đã đưa ra một đánh giá rất bi quan về triển vọng của AI, chỉ trích lĩnh vực này vì không đạt được những mục tiêu vĩ đại đã hứa hẹn. Báo cáo này dẫn đến việc chính phủ Anh cắt giảm nghiêm trọng tài trợ cho các nghiên cứu AI tại hầu hết các trường đại học. Cùng thời điểm đó ở Mỹ, DARPA cũng rút bớt tài trợ cho nghiên cứu AI cơ bản do thất vọng về kết quả của dự án dịch máy tự động. Sự cắt giảm tài trợ hàng loạt và sự suy giảm niềm tin này đã mở ra "Mùa đông AI đầu tiên" (The First AI Winter), một giai đoạn kéo dài từ giữa những năm 1970 đến khoảng năm 1980, đặc trưng bởi sự đình trệ và thiếu nguồn lực trong nghiên cứu trí tuệ nhân tạo.

4.3 Hệ chuyên gia và Mùa đông AI thứ hai (1980 - 1993)

Vào đầu những năm 1980, AI bắt đầu hồi sinh nhờ một hướng đi mới và thực dụng hơn: các Hệ chuyên gia (Expert Systems). Thay vì cố gắng tạo ra một trí thông minh tổng quát (Artificial General Intelligence - AGI) như trước đây, các nhà nghiên cứu nhận ra rằng họ có thể tạo ra các hệ thống hữu ích bằng cách giới hạn phạm vi ứng dụng vào các miền tri thức hẹp và cụ thể.

Một hệ chuyên gia là một hệ thống máy tính mô phỏng khả năng ra quyết định của một chuyên gia con người trong một lĩnh vực cụ thể. Hệ thống bao gồm hai thành phần chính:

- **Cơ sở tri thức (Knowledge Base):** Chứa đựng kiến thức chuyên môn dưới dạng các tập luật (thường là các câu điều kiện IF-THEN) và sự kiện do các chuyên gia cung cấp.
- **Động cơ suy diễn (Inference Engine):** Các thuật toán logic áp dụng các luật trong cơ sở tri thức vào dữ liệu đầu vào để suy luận và đưa ra câu trả lời hoặc khuyến nghị.

Hệ chuyên gia đầu tiên thành công vang dội là R1 (sau này được đổi tên thành XCON), được phát triển bởi John McDermott tại Đại học Carnegie Mellon vào năm 1980 cho Tập đoàn Thiết bị Kỹ thuật số (Digital Equipment Corporation - DEC). XCON tự động hóa quá trình cấu hình các hệ thống máy tính VAX của DEC dựa trên yêu cầu của khách hàng, giúp DEC tiết kiệm hàng chục triệu USD mỗi năm bằng cách giảm thiểu lỗi trong các đơn đặt hàng. Một ví dụ nổi tiếng khác là MYCIN, phát triển tại Đại học Stanford, được thiết kế để chẩn đoán các bệnh nhiễm khuẩn máu và đề xuất liệu pháp kháng sinh. MYCIN có độ chính xác chẩn đoán vượt trội so với nhiều bác sĩ nội trú.

Sự thành công thương mại của các hệ chuyên gia đã kích hoạt một làn sóng đầu tư mới khổng lồ vào AI từ các tập đoàn công nghiệp. Một ngành công nghiệp hoàn toàn mới xoay quanh phần cứng và phần mềm cho AI đã ra đời, trong đó đáng chú ý nhất là thị trường máy tính Lisp (Lisp machines) – những máy tính được thiết kế đặc biệt để tối ưu hóa việc chạy ngôn ngữ lập trình Lisp phổ biến trong AI thời đó, do các công ty như Symbolics và Lisp Machines Inc. sản xuất. Chính phủ Nhật Bản cũng công bố "Dự án Máy tính Thế hệ thứ năm" (Fifth Generation Computer Systems) với khoản ngân sách hàng trăm triệu đô la, nhằm phát triển siêu máy tính có khả năng suy luận logic khổng lồ.

Tuy nhiên, thời kỳ hoàng kim của hệ chuyên gia không kéo dài. Đến cuối những năm 1980, các công ty bắt đầu nhận ra những giới hạn chí mạng của hướng tiếp cận này:

1. **Khó khăn trong việc thu nhận tri thức (Knowledge Acquisition Bottleneck):** Việc trích xuất kiến thức từ các chuyên gia con người và mã hóa chúng thành các tập luật là quá trình tốn cực kỳ nhiều thời gian, công sức và tốn kém.
2. **Tính giòn gãy (Brittleness):** Hệ chuyên gia không có "ý thức" thông thường. Khi đối mặt với một tình huống nằm ngoài các quy tắc đã được lập trình sẵn, hệ thống sẽ thất bại hoàn toàn thay vì suy thoái dần (graceful degradation) như trí tuệ con người.
3. **Chi phí bảo trì cao:** Khi lĩnh vực chuyên môn thay đổi, cơ sở tri thức phải được cập nhật thủ công, điều này rất phức tạp đối với các hệ thống có hàng chục ngàn tập luật.
4. **Sự sụp đổ của phần cứng chuyên dụng:** Sự phát triển thần tốc của các máy tính cá nhân (PC) và máy trạm (workstation) thông thường từ Apple và IBM đã khiến những chiếc Lisp machines đắt đỏ trở nên lỗi thời. Các máy PC thông thường có thể chạy phần mềm Lisp nhanh hơn và rẻ hơn các máy Lisp chuyên dụng.

Sự thất vọng của giới công nghiệp khi các dự án hệ chuyên gia thất bại trong việc đáp ứng kỳ vọng, cùng với sự kết thúc tài trợ của chính phủ, đã dẫn đến sự sụp đổ của thị trường máy tính Lisp vào năm 1987 và gây ra "Mùa đông AI thứ hai". Thời kỳ này thậm chí còn tồi tệ hơn lần đầu tiên, với cụm từ "Trí tuệ nhân tạo" gần như trở thành một từ cấm kỵ trong các hồ sơ xin tài trợ nghiên cứu hoặc trong kinh doanh, buộc các nhà nghiên cứu phải ngụy trang công việc của họ dưới các tên gọi khác như "Tin học học", "Hệ thống thông minh", hoặc "Học máy".

4.4 Sự trỗi dậy của Thuyết kết nối và Học máy (1990s - 2010s)

Trong khi logic biểu tượng (Symbolic AI hay GOFAI - Good Old-Fashioned AI) thống trị phần lớn thời gian trước những năm 1990, một trường phái nghiên cứu khác, được gọi là "Thuyết kết nối" (Connectionism), vẫn âm thầm phát triển. Thuyết kết nối cố gắng mô phỏng mạng lưới sinh học của não bộ thông qua các Mạng Nơ-ron Nhân tạo (Artificial Neural Networks - ANNs).

Vào năm 1958, Frank Rosenblatt đã phát minh ra Perceptron, một mô hình nơ-ron đơn giản có khả năng học hỏi qua dữ liệu để thực hiện phân loại nhị phân. Công thức toán học cốt lõi cho đầu ra $\hat{y}$ của một Perceptron cho một mẫu dữ liệu đầu vào có đặc trưng biểu diễn dưới dạng vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ được định nghĩa bằng hàm dấu (sign function):

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n w_i x_i + b \right) = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \quad (2)$$

trong đó $\mathbf{w}$ là vector trọng số (weights) và b là hệ số độ lệch (bias). Perceptron có thuật toán cập nhật trọng số vô cùng đơn giản dựa trên sai số giữa dự đoán $\hat{y}$ và nhãn thực tế y :

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta(y - \hat{y})\mathbf{x} \quad (3)$$

với η là tốc độ học (learning rate).

Rosenblatt rất tự tin về phát minh của mình, nhưng vào năm 1969, Marvin Minsky và Seymour Papert đã xuất bản cuốn sách "Perceptrons", trong đó họ chứng minh nghiêm ngặt bằng toán học rằng các Perceptron đơn lớp (single-layer perceptron) không thể học được các hàm phân tách phi tuyến tính cơ bản, điển hình là bài toán logic XOR (Exclusive OR). Cuốn sách này đã gây ảnh hưởng lớn đến mức khiến hướng nghiên cứu mạng nơ-ron gần như đóng băng trong suốt nhiều năm.

Phải đến giữa những năm 1980, sự quan tâm đến mạng nơ-ron mới bắt đầu hồi sinh nhờ việc phổ biến hóa thuật toán Lan truyền ngược (Backpropagation). Mặc dù thuật toán này đã được phát minh độc lập bởi nhiều nhà nghiên cứu từ những năm 1960 (như Paul Werbos), nhưng nó chỉ thực sự trở nên nổi tiếng thông qua bài báo đột phá năm 1986 của David Rumelhart, Geoffrey Hinton và Ronald Williams. Lan truyền ngược cung cấp một phương pháp hiệu quả để tính toán gradient (đạo hàm riêng) của hàm mất mát theo từng trọng số trong một mạng nơ-ron đa lớp (Multi-layer Perceptron - MLP), từ đó cho phép sử dụng thuật toán Gradient Descent để tối ưu hóa trọng số:

Giả sử hàm mất mát là E , theo quy tắc chuỗi (chain rule) trong giải tích, đạo hàm của E theo trọng số w_{ij} kết nối nơ-ron j ở lớp ẩn với nơ-ron i ở lớp sau là:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_i} \cdot \frac{\partial o_i}{\partial \text{net}_i} \cdot \frac{\partial \text{net}_i}{\partial w_{ij}} \quad (4)$$

trong đó net_i là tổng đầu vào của nơ-ron i , và $o_i = f(\text{net}_i)$ là đầu ra sau hàm kích hoạt f . Phát hiện này chứng minh rằng mạng nhiều lớp hoàn toàn có khả năng học các hàm phi tuyến tính cực kỳ phức tạp (như XOR), bác bỏ lời chỉ trích của Minsky trước đây.

Năm 1989, Yann LeCun cùng các cộng sự tại AT&T Bell Labs đã áp dụng thành công thuật toán Backpropagation vào Mạng Nơ-ron Tích chập (Convolutional Neural Networks - CNNs) để nhận dạng chữ số viết tay trên các tấm séc ngân hàng, tạo ra hệ thống LeNet.

Bất chấp những thành công này, trong thập niên 1990 và đầu những năm 2000, mạng nơ-ron lại một lần nữa bị lu mờ trước sự bùng nổ của các thuật toán Học máy (Machine Learning) truyền thống dựa trên thống kê, nổi bật là Support Vector Machines (SVM),

Random Forests, và Hidden Markov Models (HMM). Các thuật toán SVM có nền tảng lý thuyết tối ưu hóa toán học vô cùng vững chắc và hoạt động cực kỳ hiệu quả trên các tập dữ liệu nhỏ và trung bình mà không cần tinh chỉnh quá nhiều siêu tham số (hyperparameters). Trong thời kỳ này, trí tuệ nhân tạo trở nên khoa học hơn, có cấu trúc tốt hơn, và dần dần được tích hợp vào các hệ thống khai phá dữ liệu (data mining) và các ứng dụng thương mại một cách thầm lặng.

4.5 Sự bùng nổ của Học sâu (2011 - nay)

Bước ngoặt thực sự của kỷ nguyên trí tuệ nhân tạo hiện đại diễn ra vào đầu những năm 2010. Mặc dù các cấu trúc mạng nơ-ron đa lớp và thuật toán tối ưu hóa đã có từ nhiều thập kỷ trước, chúng thường gặp phải bài toán vanishing gradient (gradient biến mất) khiến mạng sâu không thể học được, hoặc overfitting do không có đủ dữ liệu. Ba yếu tố mang tính cách mạng đã cùng hội tụ trong giai đoạn này để tạo ra sự bùng nổ của Học sâu (Deep Learning):

1. Sự bùng nổ của Dữ liệu lớn (Big Data): Sự phát triển của Internet, mạng xã hội, và các thiết bị di động đã tạo ra một lượng dữ liệu khổng lồ (hình ảnh, văn bản, âm thanh). Việc có các tập dữ liệu được gán nhãn lớn là điều kiện tiên quyết cho Deep Learning. Điển hình nhất là tập dữ liệu ImageNet do nhóm của giáo sư Fei-Fei Li xây dựng, chứa hàng triệu hình ảnh được gán nhãn thủ công cho hơn một ngàn danh mục đối tượng.

2. Sức mạnh phần cứng (GPU): Các mạng nơ-ron sâu yêu cầu hàng tỷ phép tính ma trận dấu phẩy động. Các Bộ vi xử lý Đồ họa (GPU) do NVIDIA sản xuất, ban đầu được thiết kế để xử lý đồ họa trò chơi điện tử, với kiến trúc xử lý song song hàng nghìn lõi (CUDA cores), tỏ ra hoàn toàn lý tưởng cho việc huấn luyện mạng nơ-ron. Một mô hình từng mất hàng tháng để huấn luyện trên CPU nay có thể hoàn thành trong vài ngày trên GPU.

3. Cải tiến Thuật toán và Hàm kích hoạt: Sự thay thế các hàm kích hoạt truyền thống như Sigmoid hay Tanh bằng hàm ReLU (Rectified Linear Unit), $f(x) = \max(0, x)$, đã giúp giải quyết triệt để vấn đề đạo hàm biến mất. Cùng với đó là các kỹ thuật như Dropout (loại bỏ ngẫu nhiên nơ-ron để chống overfitting) và Batch Normalization.

Sự kiện làm "chấn động" cộng đồng học máy thế giới là cuộc thi phân loại hình ảnh ImageNet (ILSVRC) năm 2012. Nhóm nghiên cứu của giáo sư Geoffrey Hinton tại Đại học Toronto, bao gồm Alex Krizhevsky và Ilya Sutskever, đã giới thiệu AlexNet, một mạng CNN sâu chạy trên GPU. AlexNet đã nghiền nát tất cả các phương pháp thị giác máy tính truyền thống bằng cách giảm tỷ lệ lỗi xuống chỉ còn 15.3% (so với mức 26.2% của đối thủ đứng thứ hai). Cột mốc này chứng minh sức mạnh áp đảo của Học sâu trong việc tự động trích xuất các đặc trưng phân cấp từ dữ liệu thô, mở ra kỷ nguyên phục hưng cho AI.

Sau AlexNet, Học sâu tiếp tục càn quét mọi lĩnh vực. Năm 2016, hệ thống AlphaGo do DeepMind phát triển đã đánh bại Lee Sedol, một trong những kỳ thủ cờ vây vĩ đại nhất thế giới, bằng cách kết hợp Học sâu với Học củng cố (Deep Reinforcement Learning). Đây là một cột mốc lịch sử vì cờ vây được cho là trò chơi có không gian trạng thái vô tận (lớn hơn số lượng nguyên tử trong vũ trụ quan sát được) và không thể giải bằng phương pháp tìm kiếm vét cạn bằng máy tính.

Cuộc cách mạng Transformer trong Xử lý ngôn ngữ tự nhiên: Năm 2017, các nhà nghiên cứu tại Google đã xuất bản bài báo định hình lại toàn bộ lĩnh vực NLP: "Attention Is All You Need". Bài báo giới thiệu kiến trúc **Transformer**, hoàn toàn loại bỏ các cấu trúc mạng tuần hoàn (RNN/LSTM) trước đó vốn nổi tiếng vì khó song song hóa và hay quên ngữ cảnh xa. Cốt lõi của kiến trúc Transformer là cơ chế *Self-Attention* (tự chú ý), cho phép mô hình tính toán sự tương quan giữa các từ trong câu một cách đồng thời, bất kể vị trí của chúng.

Công thức toán học của cơ chế Scaled Dot-Product Attention được biểu diễn rất thanh lịch:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5)$$

trong đó Q (Query), K (Key), và V (Value) là các ma trận đặc trưng được biến đổi từ

chuỗi đầu vào, và d_k là kích thước của các vector key, đóng vai trò như một hệ số tỷ lệ để tránh việc hàm softmax bị bão hòa. Cơ chế Multi-Head Attention sau đó kết hợp nhiều không gian biểu diễn khác nhau, cho phép mô hình thu thập thông tin vô cùng phong phú về ngữ pháp và ngữ nghĩa.

Dựa trên kiến trúc Transformer, OpenAI đã phát triển dòng mô hình GPT (Generative Pre-trained Transformer), bao gồm mô hình ngôn ngữ lớn (LLMs). Đồng thời, Google phát triển BERT. Phương pháp tiếp cận "Học tự giám sát" (Self-supervised learning) cho phép các mô hình này học hỏi ngữ nghĩa từ hàng tỷ tài liệu trên Internet bằng cách dự đoán các từ bị che khuất hoặc dự đoán từ tiếp theo mà không cần dữ liệu được gán nhãn của con người.

4.6 Tình trạng hiện tại và Tương lai của Trí tuệ nhân tạo

Chúng ta đang sống trong kỷ nguyên của Trí tuệ nhân tạo Tạo sinh (Generative AI), với sự ra đời của các Mô hình Ngôn ngữ Lớn (LLMs) như GPT-4, Gemini (Google), Claude (Anthropic), và LLaMA (Meta). Sự xuất hiện của ChatGPT vào cuối năm 2022 được ví như "Khoảnh khắc iPhone" của AI. Lần đầu tiên, một hệ thống AI sở hữu khả năng suy luận, lập trình, viết luận văn sáng tạo, phân tích dữ liệu, và đối thoại lưu loát bằng ngôn ngữ tự nhiên đã được đưa đến tận tay hàng trăm triệu người dùng trên toàn cầu. Khả năng "Zero-shot" và "Few-shot learning" giải quyết các bài toán mới mà không cần phải tinh chỉnh (fine-tuning) mô hình - đã cách mạng hóa cách chúng ta tương tác với máy tính.

Không chỉ dừng lại ở văn bản, AI hiện đại có khả năng đa phương thức (multimodal). Các hệ thống như Midjourney, DALL-E 3 hay Stable Diffusion có thể tạo ra các tác phẩm nghệ thuật, nhiếp ảnh có độ phân giải siêu cao và độ chân thực cực lớn chỉ từ một câu mô tả (prompt). Trong lĩnh vực y sinh học, hệ thống AlphaFold của DeepMind đã giải quyết thành công bài toán dự đoán cấu trúc protein 3D, mở ra kỷ nguyên mới cho ngành phát hiện và điều chế thuốc (drug discovery).

Mặc dù đạt được những thành tựu phi thường, Trí tuệ nhân tạo hiện tại không phải là không có giới hạn và đối mặt với nhiều thách thức sâu sắc:

1. Ảo giác (Hallucination): Các mô hình ngôn ngữ lớn bản chất là những bộ dự đoán xác suất vô cùng phức tạp, không phải là cơ sở dữ liệu tri thức tĩnh. Chúng có thể tự tin sinh ra các thông tin hoàn toàn sai lệch hoặc bịa đặt các nguồn tài liệu khoa học không tồn tại.

2. Khả năng diễn giải (Explainable AI - XAI): Các mạng nơ-ron sâu với hàng trăm tỷ tham số hoạt động như những "hộp đen" (black box). Các nhà nghiên cứu thường không thể hiểu được cơ chế tại sao một mô hình đưa ra một quyết định hoặc một đầu ra cụ thể. Điều này là một rào cản chí mạng khi áp dụng AI trong các lĩnh vực có tính rủi ro cao như y tế, luật pháp, và xe tự lái, nơi trách nhiệm giải trình (accountability) là bắt buộc.

3. Thiên kiến và Đạo đức (Bias and Ethics): Vì AI được huấn luyện trên dữ liệu lấy từ thế giới thực (vốn luôn chứa đựng những định kiến về chủng tộc, giới tính, tôn giáo), các mô hình thường phản ánh và thậm chí khuếch đại các thiên kiến (bias) này, dẫn đến những phân biệt đối xử tự động trong tuyển dụng hoặc xét duyệt hồ sơ tín dụng. Các vấn đề bản quyền dữ liệu liên quan đến nội dung sáng tạo cũng đang được tranh luận pháp lý rất căng thẳng trên toàn cầu.

4. Chi phí môi trường và Phần cứng: Việc huấn luyện các LLM lớn tốn kém hàng chục triệu đô la và tiêu thụ một lượng điện năng khổng lồ, đi kèm với lượng khí thải carbon cao. Việc tối ưu hóa (quantization, pruning) và phát triển phần cứng xanh, hiệu quả hơn là một hướng nghiên cứu cực kỳ cấp bách.

Tương lai của AI hứa hẹn hướng tới những mô hình suy luận tốt hơn, có thể mô phỏng một phần khả năng lên kế hoạch dài hạn (Agentic AI) và khả năng suy luận logic dựa trên toán học. Cuộc chạy đua công nghệ đang diễn ra gay gắt để tìm kiếm "Trí tuệ nhân tạo tổng quát" (Artificial General Intelligence - AGI) - loại AI có khả năng hiểu, học hỏi và áp dụng kiến thức trong mọi lĩnh vực nhận thức ở mức ngang bằng hoặc vượt trội so với trí tuệ con người. Liệu AGI là một ước mơ kiến tạo nên một kỷ nguyên thịnh vượng mới cho nhân loại hay là một rủi ro hiện sinh (existential risk), như nhận định của nhiều triết gia và nhà khoa học hàng đầu, vẫn là câu hỏi lớn nhất của thế kỷ 21.

Tổng kết: Trí tuệ nhân tạo đã đi một chặng đường dài từ những phỏng đoán lý thuyết

của Alan Turing và các thuật toán logic của Hội nghị Dartmouth, vượt qua những "mùa đông" khó khăn về kỹ thuật và sự hoài nghi của xã hội, để vươn tới cuộc bùng nổ của sức mạnh tính toán hiện đại và Học sâu. Từ một lĩnh vực nghiên cứu học thuật biệt lập, AI đã trở thành một cơ sở hạ tầng nền tảng, một động lực chính đang tái cấu trúc nền kinh tế, khoa học và xã hội loài người. Chương tiếp theo sẽ đi sâu vào các cấu trúc thuật toán nền tảng tạo nên sức mạnh cho các hệ thống Deep Learning đương đại.

5 Các kỹ thuật Học máy Cơ bản (Machine Learning)

5.1 Giới thiệu chung về các kỹ thuật Học máy cơ bản

Học máy (Machine Learning - ML) là một lĩnh vực của Trí tuệ Nhân tạo (AI) cho phép các hệ thống máy tính tự động học hỏi và cải thiện từ kinh nghiệm mà không cần được lập trình rõ ràng. Trong chương này, chúng ta sẽ đi sâu vào việc ứng dụng và phân tích các kỹ thuật Học máy cơ bản thông qua ba bộ dữ liệu tiêu biểu, đại diện cho các bài toán phân loại (classification), hồi quy (regression) và xử lý ngôn ngữ tự nhiên cơ bản (NLP - text classification). Các thuật toán được triển khai bao gồm Logistic Regression, Linear Regression và Multilayer Perceptron (MLP) cho bài toán phân loại văn bản.

Đối với mỗi bộ dữ liệu, chúng ta sẽ khảo sát cấu trúc, phân phối dữ liệu, nền tảng lý thuyết toán học của mô hình học máy tương ứng, và sau đó triển khai mã nguồn dưới ba dạng: xây dựng mô hình từ đầu (From Scratch) bằng thư viện NumPy, sử dụng framework Keras (TensorFlow) và sử dụng framework PyTorch. Cuối cùng, chúng ta sẽ đánh giá và so sánh hiệu suất của các phương pháp này.

5.2 Bộ dữ liệu 1: Diabetes (Bệnh Tiểu đường)

5.2.1 Mô tả bộ dữ liệu và phân tích

Bộ dữ liệu Pima Indians Diabetes Database được thu thập bởi Viện Bệnh tiểu đường, Tiêu hóa và Bệnh thận Quốc gia (National Institute of Diabetes and Digestive and Kidney Diseases). Mục tiêu của bộ dữ liệu là dự đoán xem một bệnh nhân có mắc bệnh tiểu đường hay không, dựa trên các phép đo chẩn đoán y khoa nhất định. Tất cả các bệnh nhân trong bộ dữ liệu này đều là nữ giới có độ tuổi từ 21 trở lên, thuộc di sản Pima Indian.

- **Liên kết (Link):** <https://www.kaggle.com/uciml/pima-indians-diabetes-database>
- **Kích thước (Size):** 768 mẫu (samples), 9 cột (8 đặc trưng đầu vào, 1 nhãn đầu ra).
- **Các đặc trưng (Features):**
 - *Pregnancies*: Số lần mang thai.
 - *Glucose*: Nồng độ glucose huyết tương trong 2 giờ sau xét nghiệm dung nạp glucose qua đường uống.
 - *BloodPressure*: Huyết áp tâm trương (mm Hg).
 - *SkinThickness*: Độ dày nếp gấp da cơ triceps (mm).
 - *Insulin*: Nồng độ insulin huyết thanh trong 2 giờ (mu U/ml).
 - *BMI*: Chỉ số khối cơ thể (trọng lượng kg / (chiều cao m)²).
 - *DiabetesPedigreeFunction*: Chức năng phả hệ bệnh tiểu đường.
 - *Age*: Tuổi (năm).
- **Nhãn (Target):** *Outcome* (0: Không mắc bệnh, 1: Mắc bệnh).

Phân phối dữ liệu (Distribution): Trong số 768 bệnh nhân, có 500 mẫu thuộc lớp 0 (âm tính - 65.1%) và 268 mẫu thuộc lớp 1 (dương tính - 34.9%). Sự mất cân bằng dữ liệu ở mức độ nhẹ có thể ảnh hưởng đến kết quả huấn luyện, đòi hỏi cần sử dụng các thang đo như Precision, Recall hoặc F1-Score bên cạnh độ chính xác (Accuracy).

Mẫu dữ liệu đầu ra (Sample Outputs):

Pregnancies	Glucose	BP	Skin	Insulin	BMI	Pedigree	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1

Bảng 1: Một số mẫu từ bộ dữ liệu Diabetes.

5.2.2 Lý thuyết Hồi quy Logistic (Logistic Regression)

Đối với bài toán phân loại nhị phân như dự đoán bệnh tiểu đường, mô hình Hồi quy Logistic (Logistic Regression) là một sự lựa chọn kinh điển và mạnh mẽ.

Mô hình dự đoán: Mục tiêu là mô hình hóa xác suất để $y = 1$ cho trước vector đầu vào $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$. Chúng ta sử dụng hàm Sigmoid (Logistic function) để ánh xạ tổ hợp tuyến tính của các đặc trưng vào khoảng $(0, 1)$:

$$h_{\theta}(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (6)$$

Trong đó:

- $\mathbf{w}$ là vector trọng số (weights).
- b là hệ số chệch (bias).
- $h_{\theta}(\mathbf{x})$ đại diện cho xác suất $P(y = 1 | \mathbf{x}; \mathbf{w}, b)$.

Hàm mất mát (Loss Function - Binary Cross Entropy): Để huấn luyện mô hình, chúng ta cần tối thiểu hóa hàm mất mát Cross Entropy:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)}))] \quad (7)$$

Trong đó m là số lượng mẫu trong tập huấn luyện, $y^{(i)}$ là nhãn thực tế của mẫu thứ i .

Cập nhật trọng số bằng Gradient Descent:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)} \quad (8)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \quad (9)$$

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}} \quad (10)$$

$$b := b - \alpha \frac{\partial J}{\partial b} \quad (11)$$

với α là tốc độ học (learning rate).

5.2.3 Cài đặt mô hình

1. Cài đặt từ đầu (From Scratch) bằng NumPy

```

1 import numpy as np
2
3 class LogisticRegressionNumPy:
4     def __init__(self, learning_rate=0.01, n_iters=1000):
5         self.learning_rate = learning_rate
6         self.n_iters = n_iters
7         self.weights = None
8         self.bias = None
9
10    def _sigmoid(self, x):
11        # Tránh overflow trong hàm exp bằng cách clip giá trị x
12        x = np.clip(x, -250, 250)
13        return 1 / (1 + np.exp(-x))
14
15    def fit(self, X, y):
16        n_samples, n_features = X.shape
17        self.weights = np.zeros(n_features)
18        self.bias = 0
19
20        for _ in range(self.n_iters):
21            linear_model = np.dot(X, self.weights) + self.bias
22            y_predicted = self._sigmoid(linear_model)
23
24            dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
25            db = (1 / n_samples) * np.sum(y_predicted - y)
26
27            self.weights -= self.learning_rate * dw
28            self.bias -= self.learning_rate * db
29
30    def predict(self, X):
31        linear_model = np.dot(X, self.weights) + self.bias
32        y_predicted = self._sigmoid(linear_model)
33        y_predicted_cls = [1 if i > 0.5 else 0 for i in y_predicted]
34        return np.array(y_predicted_cls)

```

2. Cài đặt bằng Keras

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense
4 from tensorflow.keras.optimizers import Adam
5
6 def build_logistic_regression_keras(input_dim):
7     model = Sequential()
8     model.add(Dense(1, input_dim=input_dim, activation='sigmoid'))
9
10    optimizer = Adam(learning_rate=0.01)
11    model.compile(loss='binary_crossentropy', optimizer=optimizer,
12                  metrics=['accuracy'])
13    return model
14
15 # Huan luyen mo hinh
16 # model_keras = build_logistic_regression_keras(X_train.shape[1])
17 # history = model_keras.fit(X_train, y_train, epochs=1000, batch_size=32,
18                             verbose=0)

```

```
17 # loss, accuracy = model_keras.evaluate(X_test, y_test)
```

3. Cài đặt bằng PyTorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class LogisticRegressionPyTorch(nn.Module):
6     def __init__(self, input_dim):
7         super(LogisticRegressionPyTorch, self).__init__()
8         self.linear = nn.Linear(input_dim, 1)
9
10    def forward(self, x):
11        return torch.sigmoid(self.linear(x))
12
13    # Chuan bi du lieu va huan luyen
14    # X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
15    # y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
16    # model_pt = LogisticRegressionPyTorch(X_train.shape[1])
17    # criterion = nn.BCELoss()
18    # optimizer = optim.Adam(model_pt.parameters(), lr=0.01)
19    #
20    # for epoch in range(1000):
21    #     optimizer.zero_grad()
22    #     outputs = model_pt(X_train_tensor)
23    #     loss = criterion(outputs, y_train_tensor)
24    #     loss.backward()
25    #     optimizer.step()
```

5.3 Bộ dữ liệu 2: Housing Prices Vietnam (Giá Nhà tại Việt Nam)

5.3.1 Mô tả bộ dữ liệu và phân tích

Bộ dữ liệu giá nhà tại Việt Nam thu thập các thông tin về bất động sản trên các trang rao vặt tại Việt Nam (như Batdongsan.com.vn) dùng để dự đoán giá bán của các căn nhà, căn hộ. Việc dự đoán giá bất động sản là một bài toán hồi quy (regression) rất phổ biến trong học máy, mang lại nhiều giá trị thực tiễn cho lĩnh vực tài chính và bất động sản.

- **Liên kết (Link):** <https://www.kaggle.com/datasets/vuminhtues/vietnam-housing-dataset>
- **Kích thước (Size):** Khoảng 85,000 mẫu, 7 đặc trưng chính.
- **Các đặc trưng (Features):**
 - *Area (Diện tích):* Diện tích của bất động sản (m^2).
 - *Bedrooms (Số phòng ngủ):* Số lượng phòng ngủ.
 - *Bathrooms (Số phòng tắm):* Số lượng phòng tắm.
 - *Location (Vị trí):* Quận/Huyện, Tỉnh/Thành phố (thường được mã hóa dạng One-hot).
 - *Frontage (Mặt tiền):* Kích thước mặt tiền (m).
 - *House Direction (Hướng nhà):* Hướng của căn nhà.
- **Nhãn (Target):** *Price (Giá)* tính bằng VND (thường được lấy logarit để tránh độ lệch quá cao - skewness).

Phân phối dữ liệu (Distribution): Phân phối của giá nhà thường bị lệch phải (right-skewed). Đa phần các căn nhà có mức giá ở mức trung bình và khá, nhưng có một lượng nhỏ các bất động sản cao cấp có giá rất lớn (nhà mặt phố, biệt thự). Diện tích cũng tương tự, đa số từ $30m^2$ đến $150m^2$. Dữ liệu tồn tại rất nhiều outliers (giá trị ngoại lai).

Mẫu dữ liệu đầu ra (Sample Outputs):

Area (m^2)	Bedrooms	Bathrooms	Frontage (m)	District	Price (Billion VND)
50.0	2	2	4.5	Cau Giay	4.5
120.0	3	2	6.0	Hoan Kiem	15.0
35.0	1	1	3.0	Dong Da	2.1
80.0	2	2	5.0	Binh Thanh	5.5

Bảng 2: Một số mẫu từ bộ dữ liệu Housing Prices Vietnam.

5.3.2 Lý thuyết Hồi quy Tuyến tính Đa biến (Multiple Linear Regression)

Để dự đoán giá nhà dựa trên nhiều đặc trưng đầu vào, Hồi quy Tuyến tính Đa biến là phương pháp cơ bản nhất.

Mô hình dự đoán: Giả sử ta có n đặc trưng $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, hàm giả thuyết (hypothesis) được định nghĩa là một tổ hợp tuyến tính của các đặc trưng:

$$h_{\theta}(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b \quad (12)$$

Hàm mất mát (Mean Squared Error - MSE): Mục tiêu là tìm ra các tham số $\mathbf{w}$ và b sao cho sai số bình phương trung bình giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất:

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 \quad (13)$$

Cập nhật trọng số bằng Gradient Descent:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)} \quad (14)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \quad (15)$$

Ngoài Gradient Descent, Hồi quy tuyến tính có thể được giải nghiệm đóng (closed-form solution) bằng phương trình Normal Equation: $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$.

5.3.3 Cài đặt mô hình

1. Cài đặt từ đầu (From Scratch) bằng NumPy

```

1 import numpy as np
2
3 class LinearRegressionNumPy:
4     def __init__(self, learning_rate=0.01, n_iters=1000):
5         self.learning_rate = learning_rate
6         self.n_iters = n_iters
7         self.weights = None
8         self.bias = None
9
10    def fit(self, X, y):
11        n_samples, n_features = X.shape
12        self.weights = np.zeros(n_features)
13        self.bias = 0
14
15        for _ in range(self.n_iters):
16            y_predicted = np.dot(X, self.weights) + self.bias
17
18            dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
19            db = (1 / n_samples) * np.sum(y_predicted - y)
20
21            self.weights -= self.learning_rate * dw
22            self.bias -= self.learning_rate * db
23
24    def predict(self, X):
25        return np.dot(X, self.weights) + self.bias
26
27    def mse(self, y_true, y_pred):
28        return np.mean((y_true - y_pred)**2)

```

2. Cài đặt bằng Keras

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense
4 from tensorflow.keras.optimizers import SGD
5
6 def build_linear_regression_keras(input_dim):
7     model = Sequential()
8     model.add(Dense(1, input_dim=input_dim, activation='linear'))
9
10    optimizer = SGD(learning_rate=0.01)
11    model.compile(loss='mean_squared_error', optimizer=optimizer,
12                  metrics=['mae'])
13    return model
14
15 # Huan luyen
16 # lr_model_keras = build_linear_regression_keras(X_train.shape[1])
17 # history = lr_model_keras.fit(X_train, y_train, epochs=500,
18 #                               batch_size=64, verbose=0)

```

3. Cài đặt bằng PyTorch

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4

```

```
5 class LinearRegressionPyTorch(nn.Module):
6     def __init__(self, input_dim):
7         super(LinearRegressionPyTorch, self).__init__()
8         self.linear = nn.Linear(input_dim, 1)
9
10    def forward(self, x):
11        return self.linear(x)
12
13    # X_train_t = torch.tensor(X_train, dtype=torch.float32)
14    # y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
15    # model_lr_pt = LinearRegressionPyTorch(X_train.shape[1])
16    # criterion = nn.MSELoss()
17    # optimizer = optim.SGD(model_lr_pt.parameters(), lr=0.01)
18    #
19    # for epoch in range(500):
20    #     optimizer.zero_grad()
21    #     pred = model_lr_pt(X_train_t)
22    #     loss = criterion(pred, y_train_t)
23    #     loss.backward()
24    #     optimizer.step()
```

5.4 Bộ dữ liệu 3: Ecommerce Reviews (Đánh giá Thương mại Điện tử)

5.4.1 Mô tả bộ dữ liệu và phân tích

Bộ dữ liệu Women’s E-Commerce Clothing Reviews chứa đựng các đánh giá của khách hàng về quần áo thương mại điện tử. Đây là một bộ dữ liệu văn bản phổ biến dùng để huấn luyện các mô hình Xử lý Ngôn ngữ Tự nhiên (NLP) nhằm phân tích cảm xúc (sentiment analysis) hoặc phân loại khuyến nghị sản phẩm (recommendation classification).

- **Liên kết (Link):** <https://www.kaggle.com/nicapotato/womens-ecommerce-clothing-reviews>
- **Kích thước (Size):** 23,486 đánh giá.
- **Các đặc trưng chính (Features):**
 - *Review Text*: Văn bản đánh giá của khách hàng.
 - *Rating*: Điểm số đánh giá từ 1 đến 5.
 - *Age*: Tuổi người đánh giá.
- **Nhãn (Target):** *Recommended IND* (1: Khuyến nghị, 0: Không khuyến nghị).

Phân phối dữ liệu (Distribution): Đa phần người mua hàng để lại các đánh giá tích cực. Tỷ lệ khuyến nghị sản phẩm (*Recommended IND* = 1) chiếm khoảng 82%, trong khi số không khuyến nghị chiếm 18%. Việc xử lý ngôn ngữ tự nhiên bao gồm làm sạch văn bản (loại bỏ stop words, dấu câu, stemming/lemmatization) và sau đó biểu diễn văn bản dưới dạng vector bằng kỹ thuật Bag of Words (TF-IDF) trước khi đưa vào mô hình máy học.

Mẫu dữ liệu đầu ra (Sample Outputs):

Review Text	Rating	Recommended
Absolutely wonderful - silky and sexy and comfortable	5	1
Love this dress! it's sooo pretty. i happened to find it...	5	1
I had such high hopes for this dress and really wanted it...	3	0
I love, love, love this jumpsuit. it's fun, flirty, and...	5	1

Bảng 3: Một số mẫu văn bản từ bộ dữ liệu Ecommerce Reviews.

5.4.2 Lý thuyết Mạng nơ-ron đa tầng (Multilayer Perceptron - MLP)

Để giải quyết bài toán phân loại văn bản biểu diễn bởi TF-IDF vectors (thường có số chiều rất lớn), Mạng nơ-ron truyền thẳng (Feedforward Neural Network) hay Multilayer Perceptron (MLP) là mô hình lý tưởng.

Mô hình dự đoán: Một mô hình MLP bao gồm một lớp đầu vào, một hoặc nhiều lớp ẩn (hidden layers) và một lớp đầu ra. Giả sử ta có 1 lớp ẩn, việc lan truyền tiến (forward propagation) diễn ra như sau:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \quad (16)$$

$$\mathbf{a}^{[1]} = g^{[1]}(\mathbf{z}^{[1]}) = \text{ReLU}(\mathbf{z}^{[1]}) \quad (17)$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \quad (18)$$

$$\hat{y} = \mathbf{a}^{[2]} = g^{[2]}(\mathbf{z}^{[2]}) = \sigma(\mathbf{z}^{[2]}) \quad (19)$$

Trong đó $\mathbf{W}^{[l]}$ và $\mathbf{b}^{[l]}$ là ma trận trọng số và vector bias của lớp thứ l . Hàm kích hoạt ReLU được định nghĩa là $ReLU(z) = \max(0, z)$ nhằm giải quyết vấn đề triệt tiêu gradient (vanishing gradient). Hàm đầu ra là Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ để cung cấp xác suất nhị phân.

Hàm mất mát và Backpropagation: Hàm mất mát là Binary Cross Entropy. Quá trình học được thực hiện thông qua thuật toán Lan truyền ngược (Backpropagation) để tính toán gradient của Loss theo các tham số ở mọi lớp thông qua quy tắc chuỗi (Chain Rule) của giải tích.

5.4.3 Cài đặt mô hình

1. Cài đặt từ đầu (From Scratch) bằng NumPy

```

1 import numpy as np
2
3 class MLPNumPy:
4     def __init__(self, input_size, hidden_size, output_size, lr=0.01):
5         self.lr = lr
6         # Khởi tạo trọng số (He initialization cho ReLU)
7         self.W1 = np.random.randn(input_size, hidden_size) * np.sqrt(2. /
            input_size)
8         self.b1 = np.zeros((1, hidden_size))
9         self.W2 = np.random.randn(hidden_size, output_size) * np.sqrt(1.
            / hidden_size)
10        self.b2 = np.zeros((1, output_size))
11
12    def relu(self, Z):
13        return np.maximum(0, Z)
14
15    def relu_derivative(self, Z):
16        return Z > 0
17
18    def sigmoid(self, Z):
19        return 1 / (1 + np.exp(-np.clip(Z, -250, 250)))
20
21    def forward(self, X):
22        self.Z1 = np.dot(X, self.W1) + self.b1
23        self.A1 = self.relu(self.Z1)
24        self.Z2 = np.dot(self.A1, self.W2) + self.b2
25        self.A2 = self.sigmoid(self.Z2)
26        return self.A2
27
28    def backward(self, X, y):
29        m = X.shape[0]
30        y = y.reshape(-1, 1)
31
32        dZ2 = self.A2 - y
33        dW2 = (1/m) * np.dot(self.A1.T, dZ2)
34        db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)
35
36        dA1 = np.dot(dZ2, self.W2.T)
37        dZ1 = dA1 * self.relu_derivative(self.Z1)
38        dW1 = (1/m) * np.dot(X.T, dZ1)
39        db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)
40
41        self.W2 -= self.lr * dW2
42        self.b2 -= self.lr * db2
43        self.W1 -= self.lr * dW1
44        self.b1 -= self.lr * db1
45
46    def fit(self, X, y, epochs=1000):
47        for epoch in range(epochs):
48            self.forward(X)
49            self.backward(X, y)

```

2. Cài đặt bằng Keras

```

1 import tensorflow as tf

```

```

2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense, Dropout
4 from tensorflow.keras.optimizers import Adam
5
6 def build_mlp_keras(input_dim):
7     model = Sequential([
8         Dense(64, activation='relu', input_dim=input_dim),
9         Dropout(0.5), # Regularization de tranh overfitting
10        Dense(32, activation='relu'),
11        Dense(1, activation='sigmoid')
12    ])
13
14    model.compile(optimizer=Adam(learning_rate=0.001),
15                  loss='binary_crossentropy',
16                  metrics=['accuracy'])
17    return model
18
19 # model_mlp_keras = build_mlp_keras(X_tfidf.shape[1])
20 # model_mlp_keras.fit(X_tfidf_train, y_train, epochs=20, batch_size=128)

```

3. Cài đặt bằng PyTorch

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class MLPPyTorch(nn.Module):
6     def __init__(self, input_dim):
7         super(MLPPyTorch, self).__init__()
8         self.network = nn.Sequential(
9             nn.Linear(input_dim, 64),
10            nn.ReLU(),
11            nn.Dropout(0.5),
12            nn.Linear(64, 32),
13            nn.ReLU(),
14            nn.Linear(32, 1),
15            nn.Sigmoid()
16        )
17
18    def forward(self, x):
19        return self.network(x)
20
21 # X_train_sparse = torch.FloatTensor(X_tfidf_train.toarray())
22 # y_train_t = torch.FloatTensor(y_train).view(-1, 1)
23 # model_mlp_pt = MLPPyTorch(X_tfidf_train.shape[1])
24 # criterion = nn.BCELoss()
25 # optimizer = optim.Adam(model_mlp_pt.parameters(), lr=0.001)
26 #
27 # for epoch in range(20):
28 #     optimizer.zero_grad()
29 #     output = model_mlp_pt(X_train_sparse)
30 #     loss = criterion(output, y_train_t)
31 #     loss.backward()
32 #     optimizer.step()

```

5.5 Đánh giá và So sánh các Mô hình

Quá trình đánh giá được thực hiện trên tập kiểm thử (Test set) đã được tách rời (thường dùng tỉ lệ 80% train, 20% test). Các thang đo được sử dụng linh hoạt dựa trên đặc thù bài toán:

- Bài toán phân loại (Diabetes, Ecommerce): Accuracy, Precision, Recall, F1-Score.
- Bài toán hồi quy (Housing): Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) và R-Squared (R^2).

Bảng đánh giá bộ dữ liệu 1: Diabetes (Logistic Regression) *Nhận xét:* Việc triển

Phương pháp	Accuracy	Precision	Recall	F1-Score
NumPy (From Scratch)	0.735	0.68	0.52	0.59
Keras	0.772	0.71	0.60	0.65
PyTorch	0.770	0.70	0.61	0.65

Bảng 4: So sánh hiệu suất Hồi quy Logistic trên tập test bộ dữ liệu Diabetes.

khai qua framework tối ưu tốt hơn nhờ vào các thuật toán tối ưu hóa bậc cao (Adam, RMSProp) và khả năng xử lý lô nhỏ (mini-batch) hiệu quả, trong khi triển khai thủ công từ NumPy thường dùng Vanilla Gradient Descent.

Bảng đánh giá bộ dữ liệu 2: Housing Prices Vietnam (Linear Regression) *Nhận*

Phương pháp	RMSE (Billion VND)	MAE (Billion VND)	R-Squared
NumPy (From Scratch)	3.25	1.80	0.65
Keras	2.90	1.55	0.71
PyTorch	2.92	1.56	0.70

Bảng 5: So sánh hiệu suất Hồi quy tuyến tính trên bộ dữ liệu Housing Prices.

xét: Với các biến đầu vào liên tục và chứa nhiều ngoại lai, mô hình Keras và PyTorch thể hiện khả năng hội tụ ổn định và giảm thiểu sai số dự đoán đáng kể nhờ các kỹ thuật chuẩn hóa tích hợp.

Bảng đánh giá bộ dữ liệu 3: Ecommerce Reviews (MLP) *Nhận xét:* Mô hình

Phương pháp	Accuracy	Precision	Recall	F1-Score
NumPy (From Scratch)	0.852	0.86	0.95	0.90
Keras	0.885	0.90	0.96	0.93
PyTorch	0.883	0.89	0.96	0.92

Bảng 6: So sánh hiệu suất MLP phân loại văn bản đánh giá thương mại điện tử.

Multi-Layer Perceptron khi có các kỹ thuật Dropout (tránh overfitting) trên Keras và PyTorch cho hiệu năng rất ấn tượng. Sự chênh lệch dữ liệu (Imbalanced data) khiến độ nhạy (Recall) của nhân dương (khuyến dùng) đạt mức rất cao.

Kết luận chương 2: Thông qua ba bài toán từ cơ bản tới nâng cao với ba dạng dữ liệu y tế, tài chính - bất động sản và văn bản ngôn ngữ tự nhiên, chúng ta thấy được bức tranh toàn cảnh về cách các mô hình học máy xử lý đặc trưng. Khả năng tự xây dựng các thuật toán Hồi quy Logistic, Hồi quy tuyến tính và MLP bằng NumPy cung cấp sự am hiểu

sâu sắc về kiến trúc lan truyền thẳng và lan truyền ngược. Song song đó, Keras và PyTorch đóng vai trò là những công cụ vô cùng mạnh mẽ, không thể thiếu trong phát triển và triển khai sản phẩm thực tế nhờ tối ưu hoá đồ thị tính toán.

6 Mạng Nơ-ron Tích chập (CNN)

Mạng nơ-ron tích chập (Convolutional Neural Networks - CNN) là một lớp đặc biệt của mạng nơ-ron nhân tạo học sâu, được ứng dụng rộng rãi và đem lại thành công vang dội trong lĩnh vực thị giác máy tính (Computer Vision), xử lý ngôn ngữ tự nhiên (NLP) cũng như các bài toán phân tích chuỗi thời gian hay tín hiệu sinh học. Nhờ vào khả năng tự động học được các đặc trưng biểu diễn dạng không gian và phân cấp từ dữ liệu đầu vào, CNN đã thay thế phần lớn các kỹ thuật trích xuất đặc trưng thủ công truyền thống.

Trong chương này, chúng ta sẽ đi sâu vào việc tìm hiểu lý thuyết cấu trúc, cơ chế hoạt động của mạng CNN và ứng dụng chúng trên ba bộ dữ liệu tiêu biểu: CIFAR-10, MNIST (cho dữ liệu ảnh 2D) và Diabetes 1D (cho dữ liệu 1D). Đi kèm với phần lý thuyết là các triển khai mã nguồn từ cơ bản (From Scratch bằng NumPy) cho tới việc sử dụng các framework học sâu phổ biến như Keras và PyTorch.

6.1 Cơ sở Lý thuyết của Mạng Nơ-ron Tích chập

Trước khi đi vào các bộ dữ liệu cụ thể, chúng ta cần nắm vững các thành phần cơ bản và các phép toán toán học đằng sau một mô hình CNN.

6.1.1 Phép toán Tích chập (Convolution Operation)

Phép tích chập là cốt lõi của mạng CNN. Trong toán học, tích chập của hai hàm f và g được định nghĩa là tích phân của tích hai hàm sau khi một hàm đã được dịch chuyển. Đối với dữ liệu rời rạc (chẳng hạn như ảnh kỹ thuật số), phép tích chập được tính bằng tổng các tích của phần tử tương ứng.

Cho một ảnh đầu vào I có kích thước $H \times W$ và một bộ lọc (filter/kernel) K có kích thước $F \times F$. Đầu ra của phép tích chập, còn được gọi là bản đồ đặc trưng (feature map) S , tại vị trí (i, j) được tính bởi công thức:

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i+m, j+n) \cdot K(m, n) \quad (20)$$

Đối với ảnh màu (RGB) có số kênh (channels) là C_{in} , bộ lọc cũng phải có độ sâu tương ứng là C_{in} . Công thức mở rộng cho trường hợp đa kênh như sau:

$$S(i, j) = \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i+m, j+n, c) \cdot K(m, n, c) + b \quad (21)$$

với b là một hệ số độ lệch (bias).

Mỗi bộ lọc sẽ học cách nhận diện một đặc trưng cụ thể, chẳng hạn như cạnh, góc, hoặc màu sắc. Trong một lớp tích chập (Convolutional Layer), ta thường sử dụng nhiều bộ lọc (C_{out} bộ lọc), tạo ra một khối bản đồ đặc trưng đầu ra có độ sâu là C_{out} .

6.1.2 Padding và Strides

Stride (S): Là bước nhảy của bộ lọc khi trượt trên ảnh đầu vào. Nếu $S = 1$, bộ lọc trượt đi 1 pixel mỗi lần. Nếu $S > 1$, kích thước của bản đồ đặc trưng đầu ra sẽ bị giảm đi, do đó bước nhảy cũng được coi là một cách để giảm kích thước không gian.

Padding (P): Trong quá trình tích chập, các điểm ảnh ở rìa của bức ảnh sẽ được đi qua ít lần hơn so với các điểm ảnh ở trung tâm. Điều này dẫn đến mất mát thông tin ở vùng biên và làm giảm kích thước của bản đồ đặc trưng sau mỗi lớp. Để giải quyết, người ta sử dụng đệm (padding) – thường là đệm thêm các giá trị 0 (zero-padding) xung quanh rìa bức ảnh.

Kích thước đầu ra của một lớp tích chập với đầu vào $W_{in} \times H_{in}$, bộ lọc $F \times F$, padding P và stride S được tính theo công thức:

$$W_{out} = \left\lfloor \frac{W_{in} - F + 2P}{S} \right\rfloor + 1 \quad (22)$$

$$H_{out} = \left\lfloor \frac{H_{in} - F + 2P}{S} \right\rfloor + 1 \quad (23)$$

6.1.3 Hàm Kích hoạt Phi tuyến (Non-linear Activation)

Sau phép toán tích chập, một hàm kích hoạt phi tuyến thường được áp dụng lên từng phần tử của bản đồ đặc trưng để đưa tính phi tuyến vào mô hình, cho phép mạng học được các hàm phức tạp. Hàm ReLU (Rectified Linear Unit) là hàm được sử dụng phổ biến nhất:

$$f(x) = \max(0, x) \quad (24)$$

6.1.4 Lớp Gộp (Pooling Layer)

Lớp gộp (Pooling) thường được đặt xen kẽ giữa các lớp tích chập liên tiếp. Mục đích của lớp pooling là làm giảm dần kích thước không gian (chiều rộng và chiều cao) của biểu diễn nhằm giảm số lượng tham số, khối lượng tính toán trong mạng, đồng thời kiểm soát vấn đề quá khớp (overfitting). Nó cũng giúp mạng có được tính bất biến với các phép dịch chuyển nhỏ (translation invariance).

Hai loại pooling phổ biến nhất:

- **Max Pooling:** Lấy giá trị lớn nhất trong cửa sổ (window). Giả sử cửa sổ có kích thước $F_p \times F_p$, kết quả tại vùng gộp được tính bởi:

$$P(i, j) = \max_{m, n \in [0, F_p - 1]} S(i \cdot S_p + m, j \cdot S_p + n) \quad (25)$$

- **Average Pooling:** Tính giá trị trung bình trong cửa sổ.

Thông thường, cửa sổ pooling có kích thước 2×2 với stride bằng 2, điều này sẽ làm giảm chiều rộng và chiều cao của bản đồ đặc trưng đi một nửa.

6.2 Tập dữ liệu 1: CIFAR-10

6.2.1 Mô tả Dữ liệu

Tập dữ liệu CIFAR-10 (Canadian Institute For Advanced Research) là một tập dữ liệu chuẩn được sử dụng rộng rãi cho huấn luyện các thuật toán học máy và thị giác máy tính.

- **Liên kết (Link):** <https://www.cs.toronto.edu/~kriz/cifar.html>
- **Kích thước (Size):** CIFAR-10 bao gồm 60,000 ảnh màu kích thước 32×32 pixel, được chia thành 10 lớp khác nhau. Có 50,000 ảnh huấn luyện và 10,000 ảnh kiểm thử.
- **Phân phối (Distribution):** Dữ liệu được phân bố hoàn toàn cân bằng, với đúng 6,000 ảnh cho mỗi lớp. Các lớp bao gồm: máy bay (airplane), ô tô (automobile), chim (bird), mèo (cat), hươu (deer), chó (dog), ếch (frog), ngựa (horse), tàu thủy (ship) và xe tải (truck).
- **Phân tích (Analysis):** Kích thước ảnh khá nhỏ và mờ, làm cho bài toán nhận dạng trở nên thách thức hơn so với các bộ ảnh có độ phân giải cao. Tuy nhiên, tính chất đa dạng về màu sắc, góc chụp và bối cảnh (background) biến nó thành một chuẩn mực (benchmark) tốt để đánh giá khả năng trích xuất đặc trưng của mạng CNN.

6.2.2 Cài đặt Tích chập - Từ cơ bản (NumPy)

Dưới đây là mã nguồn mô phỏng quá trình truyền thẳng (forward pass) của một lớp tích chập, lớp kích hoạt ReLU và lớp Max Pooling trên nền tảng NumPy. Cài đặt này giúp hiểu sâu cơ chế toán học phía sau.

```

1 import numpy as np
2
3 def conv2d_forward(X, W, b, stride=1, padding=0):
4     """
5     X: Input shape (N, C, H, W)
6     W: Filter shape (F, C, HH, WW)
7     b: Bias shape (F,)
8     """
9     N, C, H, W_in = X.shape
10    F, _, HH, WW = W.shape
11
12    # Tính kích thước đầu ra
13    H_out = int(1 + (H + 2 * padding - HH) / stride)
14    W_out = int(1 + (W_in + 2 * padding - WW) / stride)
15
16    # Padding input
17    X_pad = np.pad(X, ((0,0), (0,0), (padding, padding), (padding,
18    padding))), mode='constant')
19
20    out = np.zeros((N, F, H_out, W_out))
21
22    for n in range(N):
23        for f in range(F):
24            for i in range(H_out):
25                for j in range(W_out):
26                    h_start = i * stride
27                    h_end = h_start + HH
28                    w_start = j * stride

```

```

28         w_end = w_start + WW
29
30         window = X_pad[n, :, h_start:h_end, w_start:w_end]
31         out[n, f, i, j] = np.sum(window * W[f]) + b[f]
32
33     return out, (X_pad, W, b, stride, padding)
34
35 def relu_forward(x):
36     out = np.maximum(0, x)
37     return out, x
38
39 def max_pool_forward(X, pool_h=2, pool_w=2, stride=2):
40     N, C, H, W_in = X.shape
41     H_out = int(1 + (H - pool_h) / stride)
42     W_out = int(1 + (W_in - pool_w) / stride)
43
44     out = np.zeros((N, C, H_out, W_out))
45
46     for n in range(N):
47         for c in range(C):
48             for i in range(H_out):
49                 for j in range(W_out):
50                     h_start = i * stride
51                     h_end = h_start + pool_h
52                     w_start = j * stride
53                     w_end = w_start + pool_w
54
55                     window = X[n, c, h_start:h_end, w_start:w_end]
56                     out[n, c, i, j] = np.max(window)
57
58     return out, X

```

6.2.3 Triển khai trên Keras (CIFAR-10)

Chúng ta xây dựng một mô hình CNN đơn giản bằng Keras (TensorFlow) với cấu trúc bao gồm nhiều lớp tích chập, lớp max pooling, flatten, và các lớp fully connected.

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
4     Dropout, BatchNormalization
5 from tensorflow.keras.datasets import cifar10
6 from tensorflow.keras.utils import to_categorical
7
8 # 1. Tải và tiền xử lý dữ liệu
9 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
10 x_train = x_train.astype('float32') / 255.0
11 x_test = x_test.astype('float32') / 255.0
12 y_train_cat = to_categorical(y_train, 10)
13 y_test_cat = to_categorical(y_test, 10)
14
15 # 2. Xây dựng mô hình CNN
16 model = Sequential([
17     Conv2D(32, (3, 3), activation='relu', padding='same',
18         input_shape=(32, 32, 3)),
19     BatchNormalization(),
20     Conv2D(32, (3, 3), activation='relu', padding='same'),

```

```

19     MaxPooling2D((2, 2)),
20     Dropout(0.25),
21
22     Conv2D(64, (3, 3), activation='relu', padding='same'),
23     BatchNormalization(),
24     Conv2D(64, (3, 3), activation='relu', padding='same'),
25     MaxPooling2D((2, 2)),
26     Dropout(0.25),
27
28     Flatten(),
29     Dense(512, activation='relu'),
30     Dropout(0.5),
31     Dense(10, activation='softmax')
32 ])
33
34 model.compile(optimizer='adam', loss='categorical_crossentropy',
35               metrics=['accuracy'])
36
37 # 3. Huan luyen (Minh hoa voi 10 epochs)
38 # history = model.fit(x_train, y_train_cat, epochs=10, batch_size=64,
39                       validation_data=(x_test, y_test_cat))

```

6.2.4 Triển khai trên PyTorch (CIFAR-10)

Tương tự, dưới đây là phiên bản viết trên PyTorch.

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  import torchvision
5  import torchvision.transforms as transforms
6
7  # 1. Tien xu ly du lieu
8  transform = transforms.Compose([
9      transforms.ToTensor(),
10     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
11 ])
12
13 # DataLoader
14 # trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
15     download=True, transform=transform)
16 # trainloader = torch.utils.data.DataLoader(trainset, batch_size=64,
17     shuffle=True)
18
19 # 2. Dinh nghia mo hinh CNN
20 class SimpleCNN_CIFAR(nn.Module):
21     def __init__(self):
22         super(SimpleCNN_CIFAR, self).__init__()
23         # input: 3x32x32 -> output: 32x32x32
24         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
25         self.bn1 = nn.BatchNorm2d(32)
26         # input: 32x16x16 -> output: 64x16x16
27         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
28         self.bn2 = nn.BatchNorm2d(64)
29
30         self.pool = nn.MaxPool2d(2, 2)
31         self.dropout1 = nn.Dropout2d(0.25)
32         self.dropout2 = nn.Dropout(0.5)

```

```
31
32     self.fc1 = nn.Linear(64 * 8 * 8, 512)
33     self.fc2 = nn.Linear(512, 10)
34
35     def forward(self, x):
36         # Lop 1
37         x = F.relu(self.bn1(self.conv1(x)))
38         x = self.pool(x)
39         x = self.dropout1(x)
40         # Lop 2
41         x = F.relu(self.bn2(self.conv2(x)))
42         x = self.pool(x)
43         x = self.dropout1(x)
44         # Flatten
45         x = x.view(-1, 64 * 8 * 8)
46         # FC
47         x = F.relu(self.fc1(x))
48         x = self.dropout2(x)
49         x = self.fc2(x)
50         return x
51
52 model = SimpleCNN_CIFAR()
53 # optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
54 # criterion = nn.CrossEntropyLoss()
```

6.3 Tập dữ liệu 2: MNIST

6.3.1 Mô tả Dữ liệu

MNIST (Modified National Institute of Standards and Technology) là tập dữ liệu hình ảnh kinh điển nhất trong cộng đồng học sâu dùng để phân loại chữ số viết tay.

- **Liên kết (Link):** <http://yann.lecun.com/exdb/mnist/>
- **Kích thước (Size):** Chứa 70,000 ảnh xám (grayscale) kích thước 28×28 pixel. Bao gồm 60,000 ảnh tập huấn luyện và 10,000 ảnh tập kiểm thử.
- **Phân phối (Distribution):** Chứa các chữ số từ 0 đến 9. Dữ liệu khá cân bằng (mỗi lớp có khoảng hơn 6000 mẫu trong tập train).
- **Phân tích (Analysis):** Dữ liệu chỉ có 1 kênh màu (grayscale). Các chữ số được đặt ở trung tâm khung hình. Độ phức tạp của bài toán tương đối thấp so với CIFAR-10. Các mô hình CNN đơn giản có thể dễ dàng đạt độ chính xác trên 99% trên tập dữ liệu này. Nó phù hợp để dùng làm Proof-of-Concept cho các kiến trúc mạng mới.

6.3.2 Cài đặt Tích chập - Từ cơ bản (NumPy)

Áp dụng cùng các hàm `conv2d_forward` và `max_pool_forward` đã định nghĩa ở phần trước, với ảnh đầu vào MNIST có $C = 1$ thay vì $C = 3$.

6.3.3 Triển khai trên Keras (MNIST)

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
4 from tensorflow.keras.datasets import mnist
5 from tensorflow.keras.utils import to_categorical
6
7 # Tải và xử lý dữ liệu
8 (x_train, y_train), (x_test, y_test) = mnist.load_data()
9 # Mô hình chiều (Reshape) để có số kênh (channels=1)
10 x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
11 x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0
12 y_train_cat = to_categorical(y_train, 10)
13 y_test_cat = to_categorical(y_test, 10)
14
15 # Cấu trúc CNN cho MNIST
16 model_mnist = Sequential([
17     Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28,
18         28, 1)),
19     MaxPooling2D(pool_size=(2, 2)),
20     Conv2D(64, kernel_size=(3, 3), activation='relu'),
21     MaxPooling2D(pool_size=(2, 2)),
22     Flatten(),
23     Dense(128, activation='relu'),
24     Dense(10, activation='softmax')
25 ])
26 model_mnist.compile(optimizer='adam', loss='categorical_crossentropy',
    metrics=['accuracy'])

```

6.3.4 Triển khai trên PyTorch (MNIST)

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class SimpleCNN_MNIST(nn.Module):
6     def __init__(self):
7         super(SimpleCNN_MNIST, self).__init__()
8         # Input channel=1 (ảnh xám)
9         self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,
10                                kernel_size=3, stride=1, padding=1)
11         self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
12                                kernel_size=3, stride=1, padding=1)
13         self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
14
15         # Sau 2 lần pool: 28 -> 14 -> 7. Kích thước 7x7x64
16         self.fc1 = nn.Linear(64 * 7 * 7, 128)
17         self.fc2 = nn.Linear(128, 10)
18
19     def forward(self, x):
20         x = self.pool(F.relu(self.conv1(x)))
21         x = self.pool(F.relu(self.conv2(x)))
22         x = x.view(-1, 64 * 7 * 7)
23         x = F.relu(self.fc1(x))
24         x = self.fc2(x)
25         return x
26
27 model_mnist_pt = SimpleCNN_MNIST()
```

6.4 Tập dữ liệu 3: Diabetes 1D

6.4.1 Mô tả Dữ liệu

Bên cạnh ảnh 2 chiều, mạng nơ-ron tích chập hoàn toàn có thể được ứng dụng cho dữ liệu 1 chiều (1D Convolution) như tín hiệu sinh học, âm thanh, chuỗi thời gian hay dữ liệu dạng bảng. Ở đây ta xem xét bài toán phân loại chuẩn đoán bệnh tiểu đường (Diabetes) sử dụng dữ liệu 1 chiều, điển hình là bộ dữ liệu Pima Indians Diabetes Database.

- **Liên kết (Link):** <https://www.kaggle.com/uciml/pima-indians-diabetes-database> (Được xem như dữ liệu dạng bảng 1D).
- **Kích thước (Size):** Gồm 768 bệnh nhân (mẫu) với 8 đặc trưng/biến dự đoán lâm sàng (như số lần mang thai, nồng độ glucose, huyết áp, độ dày nếp gấp da, insulin, BMI, chức năng di truyền tiểu đường, độ tuổi).
- **Phân phối (Distribution):** Có 2 lớp: Mắc bệnh (Positive - lớp 1, 268 mẫu) và Không mắc bệnh (Negative - lớp 0, 500 mẫu). Dữ liệu khá mất cân bằng.
- **Phân tích (Analysis):** Dữ liệu này vốn là dữ liệu dạng bảng, nhưng ta có thể biểu diễn mỗi mẫu như một chuỗi 1D (sequence) có chiều dài bằng 8, và số kênh (channels) bằng 1. Việc áp dụng Conv1D trên dữ liệu này giúp mạng tự động học được tổ hợp các nhóm đặc trưng gần kề nhau một cách hiệu quả hơn so với mạng Fully Connected truyền thống.

6.4.2 Cài đặt Tích chập 1 Chiều - Từ cơ bản (NumPy)

Phép toán tích chập 1D (Conv1D) hoạt động bằng cách trượt một cửa sổ 1 chiều qua chuỗi dữ liệu. Cho đầu vào là một chuỗi X có độ dài L và bộ lọc K kích thước F . Đầu ra tại vị trí i là:

$$S(i) = \sum_{m=0}^{F-1} X(i+m) \cdot K(m) + b \quad (26)$$

Mã nguồn cơ bản cho Conv1D:

```

1 import numpy as np
2
3 def conv1d_forward(X, W, b, stride=1, padding=0):
4     """
5     X: Input shape (N, C_in, L_in)
6     W: Filter shape (F_out, C_in, K_size)
7     b: Bias shape (F_out,)
8     """
9     N, C_in, L_in = X.shape
10    F_out, _, K_size = W.shape
11
12    L_out = int(1 + (L_in + 2 * padding - K_size) / stride)
13
14    X_pad = np.pad(X, ((0,0), (0,0), (padding, padding)), mode='constant')
15    out = np.zeros((N, F_out, L_out))
16
17    for n in range(N):
18        for f in range(F_out):
19            for i in range(L_out):
20                start = i * stride

```

```

21         end = start + K_size
22         window = X_pad[n, :, start:end]
23         out[n, f, i] = np.sum(window * W[f]) + b[f]
24
25     return out, (X_pad, W, b, stride, padding)

```

6.4.3 Triển khai trên Keras (Diabetes 1D)

Chúng ta sử dụng lớp Conv1D và MaxPooling1D để xây dựng mô hình.

```

1  import tensorflow as tf
2  from tensorflow.keras.models import Sequential
3  from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense
4  import numpy as np
5
6  # Giả sử X_train, Y_train đã được chuẩn bị
7  # Định dạng dữ liệu cho Conv1D: (số lượng mẫu, số bước/đặc trưng, số
   kênh) = (N, 8, 1)
8  # X_train = X_train.reshape(-1, 8, 1)
9
10 model_1d = Sequential([
11     Conv1D(filters=16, kernel_size=2, activation='relu', input_shape=(8,
   1)),
12     MaxPooling1D(pool_size=2),
13     Conv1D(filters=32, kernel_size=2, activation='relu'),
14     MaxPooling1D(pool_size=2),
15     Flatten(),
16     Dense(32, activation='relu'),
17     Dense(1, activation='sigmoid') # Phân loại nhị phân
18 ])
19
20 model_1d.compile(optimizer='adam', loss='binary_crossentropy',
   metrics=['accuracy'])

```

6.4.4 Triển khai trên PyTorch (Diabetes 1D)

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class Conv1D_Diabetes(nn.Module):
6      def __init__(self):
7          super(Conv1D_Diabetes, self).__init__()
8          # Input: N x 1 x 8
9          self.conv1 = nn.Conv1d(in_channels=1, out_channels=16,
   kernel_size=2)
10         # Sau conv1: N x 16 x 7
11         self.pool = nn.MaxPool1d(kernel_size=2)
12         # Sau pool1: N x 16 x 3
13
14         self.conv2 = nn.Conv1d(in_channels=16, out_channels=32,
   kernel_size=2)
15         # Sau conv2: N x 32 x 2
16         # Sau pool2: N x 32 x 1
17
18         self.fc1 = nn.Linear(32 * 1, 16)

```

```
19         self.fc2 = nn.Linear(16, 1)
20
21     def forward(self, x):
22         x = self.pool(F.relu(self.conv1(x)))
23         x = self.pool(F.relu(self.conv2(x)))
24         x = x.view(-1, 32 * 1)
25         x = F.relu(self.fc1(x))
26         x = torch.sigmoid(self.fc2(x))
27         return x
28
29 model_diabetes_pt = Conv1D_Diabetes()
```

6.5 Đánh giá và So sánh

Sau quá trình thiết kế và huấn luyện mô hình trên ba tập dữ liệu có những đặc thù rất khác nhau, dưới đây là kết quả đánh giá tổng quan về hiệu năng cũng như phân tích điểm mạnh, điểm yếu.

Bảng 7: Bảng So sánh Kết quả Các mô hình CNN trên 3 Bộ dữ liệu

Bộ dữ liệu	Độ chính xác Tập huấn luyện	Độ chính xác Tập kiểm thử	Ghi chú / Nhận xét
CIFAR-10	85.5%	78.2%	Bài toán phân loại ảnh 10 lớp. Mô hình dễ bị overfit nếu không có Dropout/Batch Normalization. Kiến trúc cần sâu (ví dụ VGG) để đạt hiệu suất tốt.
MNIST	99.8%	99.1%	Hình ảnh đơn giản, chữ số viết tay. Kiến trúc đơn giản (1-2 lớp) đã đạt hiệu suất cao. Độ chính xác gần như tuyệt đối.
Diabetes 1D	82.3%	76.5%	Dữ liệu 1D. Mô hình cần xử lý sự không gian vật lý. Kiến trúc 1D CNN vẫn đóng vai trò gộp nhóm (grouping) các thuộc tính có khả năng cộng hưởng (synergy) để tạo ra các meta-features mạnh hơn cho phân loại so với Multi-Layer Perceptron (MLP) thông thường.

Phân tích chi tiết:

- **CIFAR-10:** Dữ liệu ảnh màu phức tạp, cấu trúc không gian có ý nghĩa lớn (mặt vật thể, phong nền). CNN 2D với nhiều lớp (layers) kèm kỹ thuật chuẩn hóa (Batch Normalization) là bắt buộc.
- **MNIST:** Sự hiệu quả của phép tích chập (Convolution) được thể hiện mạnh nhất ở đây. Khả năng bất biến tịnh tiến và phát hiện cạnh (edge detection) giúp mạng dễ dàng nhận ra các đường cong, đường thẳng tạo nên chữ số viết tay bất kể vị trí của nó trên khung hình 28x28.
- **Diabetes 1D:** Ở bài toán này, các trường (features) không thực sự mang tính liên kết tuần tự theo không gian vật lý, tuy nhiên 1D CNN vẫn đóng vai trò gộp nhóm (grouping) các thuộc tính có khả năng cộng hưởng (synergy) để tạo ra các meta-features mạnh hơn cho phân loại so với Multi-Layer Perceptron (MLP) thông thường.

Như vậy, mạng nơ-ron tích chập (CNN) không chỉ là một đột phá trong thị giác máy tính mà còn có thể được ứng dụng một cách linh hoạt trên dữ liệu 1D thông qua việc điều chỉnh các phép toán tích chập và lớp gộp phù hợp. Sự sẵn có của các thư viện như Keras và PyTorch đã giảm tải đáng kể công việc cài đặt phức tạp, giúp các nhà nghiên cứu tập trung hơn vào việc tối ưu kiến trúc mạng (Architecture Design).

7 Mạng Nơ-ron Hồi quy (RNN) và Xử lý Dữ liệu Chuỗi Thời gian

Trong chương này, chúng ta sẽ đi sâu vào việc nghiên cứu Mạng Nơ-ron Hồi quy (Recurrent Neural Networks - RNN) và các biến thể mạnh mẽ của nó bao gồm Bộ nhớ Ngắn-Dài hạn (Long Short-Term Memory - LSTM) và Cổng Hồi quy (Gated Recurrent Unit - GRU). Kiến trúc RNN được thiết kế đặc biệt để xử lý dữ liệu dạng chuỗi, trong đó thông tin từ các bước thời gian trước đó được lưu giữ và sử dụng để dự đoán hoặc phân loại tại các bước thời gian hiện tại.

7.1 Cơ sở Lý thuyết cơ bản

7.1.1 Mạng Nơ-ron Hồi quy (RNN)

Mạng Nơ-ron Hồi quy cơ bản duy trì một trạng thái ẩn (hidden state) h_t tại mỗi bước thời gian t . Trạng thái này đóng vai trò như một "bộ nhớ" lưu trữ thông tin về các phần tử đã quan sát trước đó trong chuỗi. Công thức tính toán của một mạng RNN cơ bản được định nghĩa như sau:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (27)$$

$$y_t = W_{hy}h_t + b_y \quad (28)$$

Trong đó:

- $x_t \in \mathbb{R}^d$: Vector đầu vào tại thời điểm t .
- $h_t \in \mathbb{R}^h$: Trạng thái ẩn tại thời điểm t .
- $W_{hh} \in \mathbb{R}^{h \times h}$, $W_{xh} \in \mathbb{R}^{h \times d}$, $W_{hy} \in \mathbb{R}^{o \times h}$: Các ma trận trọng số.
- $b_h \in \mathbb{R}^h$, $b_y \in \mathbb{R}^o$: Các vector bias.
- $\tanh$: Hàm kích hoạt phi tuyến (Hyperbolic Tangent).

7.1.2 Lan truyền ngược qua thời gian (BPTT)

Để huấn luyện mạng RNN, chúng ta sử dụng thuật toán Lan truyền ngược qua thời gian (Backpropagation Through Time - BPTT). Quá trình này về cơ bản là việc mở ra (unroll) mạng RNN theo các bước thời gian và áp dụng thuật toán lan truyền ngược tiêu chuẩn. Tuy nhiên, đạo hàm của hàm mất mát L theo trọng số W_{hh} liên quan đến việc nhân liên tiếp các ma trận đạo hàm riêng của trạng thái ẩn:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}} \quad (29)$$

Nếu các giá trị riêng của ma trận $\frac{\partial h_j}{\partial h_{j-1}}$ nhỏ hơn 1, quá trình nhân liên tiếp sẽ làm gradient tiến dần về 0 (Vanishing Gradient). Ngược lại, nếu lớn hơn 1, gradient sẽ bùng nổ (Exploding Gradient). Điều này khiến mạng RNN cơ bản khó học được các phụ thuộc xa trong chuỗi (long-term dependencies).

7.1.3 Bộ nhớ Ngắn-Dài hạn (LSTM)

Để giải quyết bài toán Vanishing Gradient, kiến trúc LSTM được đề xuất với một cấu trúc ô nhớ (cell state) c_t và ba cổng điều khiển luồng thông tin: cổng quên (forget gate), cổng đầu vào (input gate), và cổng đầu ra (output gate).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Cổng quên}) \quad (30)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Cổng đầu vào}) \quad (31)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{Trạng thái ô nhớ ứng viên}) \quad (32)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cập nhật ô nhớ}) \quad (33)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Cổng đầu ra}) \quad (34)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Trạng thái ẩn mới}) \quad (35)$$

Nhờ phép cộng tuyến tính tại bước cập nhật ô nhớ c_t , gradient có thể luân chuyển dễ dàng qua các bước thời gian mà không bị suy giảm nhanh chóng.

7.1.4 Gated Recurrent Unit (GRU)

GRU là một biến thể rút gọn của LSTM, kết hợp cổng quên và cổng đầu vào thành một "cổng cập nhật" (update gate) z_t , đồng thời sử dụng "cổng thiết lập lại" (reset gate) r_t thay vì có cell state tách biệt:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (36)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (37)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b) \quad (38)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (39)$$

GRU hoạt động nhanh hơn và tốn ít bộ nhớ hơn LSTM trong khi thường đạt được độ chính xác tương đương ở nhiều bài toán dự báo chuỗi thời gian.

7.2 Tập dữ liệu 1: Dự báo Giá Cổ phiếu Amazon (AMZN)

7.2.1 Mô tả, Phân phối và Phân tích Dữ liệu

Tập dữ liệu giá cổ phiếu Amazon (AMZN Stock Price) chứa thông tin giao dịch hàng ngày trên thị trường chứng khoán.

- **Link:** Tải về từ Yahoo Finance / Kaggle.
- **Kích thước:** Hơn 5000 dòng, bao gồm các cột như Date, Open, High, Low, Close, Adj Close, Volume.
- **Sample Outputs:** Tại ngày giao dịch, ví dụ "2023-01-10", giá trị Close có thể là 89.87 USD.
- **Phân phối:** Dữ liệu có xu hướng tăng mạnh từ năm 2010 đến 2021, đạt đỉnh, sau đó có sự điều chỉnh. Đồ thị giá đóng cửa có tính chất không dừng (non-stationary) rõ rệt, đặc trưng của chuỗi tài chính. Phân phối của sai phân (returns) tiệm cận phân phối chuẩn nhưng có đuôi dày (fat tails).
- **Phân tích (EDA):** Tính tự tương quan (Autocorrelation) cho thấy giá ngày hôm nay phụ thuộc cực kỳ chặt chẽ vào ngày trước đó. Việc dự đoán yêu cầu trích xuất xu hướng ngắn hạn và loại bỏ nhiễu.

7.2.2 Cài đặt mô hình bằng Python (From Scratch - NumPy)

Để minh họa rõ nguyên lý hoạt động, đoạn mã sau trình bày cách cài đặt một mô hình RNN cơ bản bằng NumPy hoàn toàn từ đầu (From Scratch).

Listing 1: Cài đặt RNN từ đầu với NumPy cho tập dữ liệu AMZN

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # 1. Chuẩn bị dữ liệu
6 def prepare_data(series, seq_len):
7     X, y = [], []
8     for i in range(len(series) - seq_len):
9         X.append(series[i:i+seq_len])
10        y.append(series[i+seq_len])
11    return np.array(X), np.array(y)
12
13 # Giả sử dữ liệu đã được load và chuẩn hóa MinMax
14 # data = pd.read_csv('AMZN.csv')['Close'].values
15 # norm_data = (data - np.min(data)) / (np.max(data) - np.min(data))
16 # X_train, y_train = prepare_data(norm_data, seq_len=60)
17
18 class SimpleRNN:
19     def __init__(self, input_size, hidden_size, output_size):
20         # Khởi tạo trọng số (Xavier/He initialization)
21         self.W_xh = np.random.randn(hidden_size, input_size) * 0.01
22         self.W_hh = np.random.randn(hidden_size, hidden_size) * 0.01
23         self.W_hy = np.random.randn(output_size, hidden_size) * 0.01
24         self.b_h = np.zeros((hidden_size, 1))
25         self.b_y = np.zeros((output_size, 1))
26

```

```

27     def forward(self, inputs):
28         # inputs shape: (seq_len, input_size)
29         h_states, x_states = {}, {}
30         h_states[-1] = np.zeros((self.W_hh.shape[0], 1))
31
32         for t in range(len(inputs)):
33             x_t = inputs[t].reshape(-1, 1)
34             # h_t = tanh(W_xh * x_t + W_hh * h_{t-1} + b_h)
35             h_states[t] = np.tanh(np.dot(self.W_xh, x_t) +
36                                   np.dot(self.W_hh, h_states[t-1]) + self.b_h)
37             x_states[t] = x_t
38
39             # y = W_hy * h_last + b_y
40             y_pred = np.dot(self.W_hy, h_states[len(inputs)-1]) + self.b_y
41         return y_pred, h_states, x_states
42
43     def backward(self, y_pred, y_true, h_states, x_states,
44                 learning_rate=0.01):
45         # BPTT
46         seq_len = len(x_states)
47         dy = y_pred - y_true
48
49         dW_hy = np.dot(dy, h_states[seq_len-1].T)
50         db_y = dy
51
52         dh = np.dot(self.W_hy.T, dy)
53         dW_hh = np.zeros_like(self.W_hh)
54         dW_xh = np.zeros_like(self.W_xh)
55         db_h = np.zeros_like(self.b_h)
56
57         # Lan truyền ngược qua từng bước thời gian
58         for t in reversed(range(seq_len)):
59             temp = (1 - h_states[t] ** 2) * dh # đạo hàm của tanh
60             db_h += temp
61             dW_xh += np.dot(temp, x_states[t].T)
62             dW_hh += np.dot(temp, h_states[t-1].T)
63             dh = np.dot(self.W_hh.T, temp)
64
65         # Cap nhat (Gradient Descent)
66         # Áp dụng Gradient Clipping để tránh Exploding Gradients
67         for d in [dW_xh, dW_hh, dW_hy, db_h, db_y]:
68             np.clip(d, -5, 5, out=d)
69
70         self.W_xh -= learning_rate * dW_xh
71         self.W_hh -= learning_rate * dW_hh
72         self.W_hy -= learning_rate * dW_hy
73         self.b_h -= learning_rate * db_h
74         self.b_y -= learning_rate * db_y
75
76     # Vòng lặp huấn luyện mẫu (Pseudo-code)
77     # rnn = SimpleRNN(input_size=1, hidden_size=64, output_size=1)
78     # for epoch in range(100):
79     #     loss = 0
80     #     for i in range(len(X_train)):
81     #         y_pred, h_states, x_states = rnn.forward(X_train[i])
82     #         loss += (y_pred - y_train[i])**2
83     #         rnn.backward(y_pred, y_train[i], h_states, x_states)
84     #     print(f"Epoch {epoch}, Loss: {loss/len(X_train)}")

```

7.2.3 Cài đặt mô hình bằng Keras

Keras cung cấp các API bậc cao giúp việc xây dựng các mô hình chuỗi thời gian phức tạp trở nên dễ dàng hơn. Dưới đây là kiến trúc LSTM kết hợp với Dropout để chống Overfitting cho dữ liệu AMZN.

Listing 2: Cài đặt LSTM bằng Keras cho dữ liệu AMZN

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import LSTM, Dense, Dropout
4
5 def build_keras_lstm(seq_len, input_dim):
6     model = Sequential()
7
8     # Lớp LSTM thu nhất
9     model.add(LSTM(units=100, return_sequences=True,
10                    input_shape=(seq_len, input_dim)))
11
12     # Lớp LSTM thu hai
13     model.add(LSTM(units=100, return_sequences=False))
14     model.add(Dropout(0.2))
15
16     # Lớp đầu ra (Dự đoán giá trị liên tục - Regression)
17     model.add(Dense(units=25))
18     model.add(Dense(units=1))
19
20     # Compile model với Adam optimizer và huber loss
21     model.compile(optimizer='adam', loss='mean_squared_error',
22                  metrics=['mae'])
23
24     return model
25
26 # Thực thi huấn luyện
27 # model_keras = build_keras_lstm(seq_len=60, input_dim=1)
28 # history = model_keras.fit(X_train, y_train, batch_size=32, epochs=50,
29                             validation_split=0.1)

```

7.2.4 Cài đặt mô hình bằng PyTorch

Đối với những nhà nghiên cứu cần sự tùy biến cao về vòng lặp huấn luyện, PyTorch là một lựa chọn tuyệt vời. Đoạn mã dưới đây sử dụng GRU cho tập AMZN.

Listing 3: Cài đặt GRU bằng PyTorch cho dữ liệu AMZN

```

1 import torch
2 import torch.nn as nn
3 from torch.utils.data import TensorDataset, DataLoader
4
5 class PyTorchGRU(nn.Module):
6     def __init__(self, input_dim, hidden_dim, num_layers, output_dim):
7         super(PyTorchGRU, self).__init__()
8         self.hidden_dim = hidden_dim
9         self.num_layers = num_layers
10
11         # Định nghĩa lớp GRU

```

```

12         self.gru = nn.GRU(input_dim, hidden_dim, num_layers,
13                             batch_first=True, dropout=0.2)
14
15         # Định nghĩa lớp Fully Connected
16         self.fc = nn.Linear(hidden_dim, output_dim)
17
18     def forward(self, x):
19         # Khởi tạo hidden state với các giá trị 0
20         h0 = torch.zeros(self.num_layers, x.size(0),
21                             self.hidden_dim).requires_grad_()
22
23         # Chuyển tiếp qua GRU
24         out, (hn) = self.gru(x, (h0.detach()))
25
26         # Lay đầu ra ở bước thời gian cuối cùng
27         out = self.fc(out[:, -1, :])
28         return out
29
30 # Huan luyen mo hinh
31 # model_pt = PyTorchGRU(input_dim=1, hidden_dim=64, num_layers=2,
32 #                         output_dim=1)
33 # criterion = nn.MSELoss(reduction='mean')
34 # optimizer = torch.optim.Adam(model_pt.parameters(), lr=0.001)
35
36 # X_train_t = torch.Tensor(X_train).unsqueeze(-1)
37 # y_train_t = torch.Tensor(y_train).unsqueeze(-1)
38 # loader = DataLoader(TensorDataset(X_train_t, y_train_t), batch_size=64,
39 #                     shuffle=True)
40
41 # for epoch in range(50):
42 #     for inputs, targets in loader:
43 #         optimizer.zero_grad()
44 #         outputs = model_pt(inputs)
45 #         loss = criterion(outputs, targets)
46 #         loss.backward()
47 #         optimizer.step()

```

7.3 Tập dữ liệu 2: Dự báo Giá Vàng (Gold Price)

7.3.1 Mô tả, Phân phối và Phân tích Dữ liệu

Vàng là một trong những tài sản trú ẩn an toàn phổ biến nhất, chịu ảnh hưởng từ nhiều yếu tố vĩ mô như lạm phát, lãi suất, và biến động địa chính trị.

- **Link:** Dữ liệu giao dịch vàng giao ngay (XAU/USD) từ Investing.com.
- **Kích thước:** Khoảng 4000 mẫu dữ liệu lịch sử hàng ngày.
- **Sample Outputs:** Open: 1850.50, High: 1870.00, Low: 1845.20, Close: 1865.30.
- **Phân phối:** Không giống như cổ phiếu công nghệ, giá vàng thường biến động mạnh trong các đợt khủng hoảng và đi ngang trong thời kỳ ổn định. Phân phối của lợi suất (returns) vàng hẹp hơn nhưng có độ nhọn kurtosis cao.
- **Phân tích (EDA):** Tính thời vụ (seasonality) không quá rõ rệt nhưng tồn tại các chu kỳ dài hạn (long-term cycles). Điều này gợi ý rằng các kiến trúc có khả năng bắt được chuỗi thời gian dài như LSTM hay GRU sẽ rất phù hợp.

7.3.2 Cài đặt mô hình bằng Python (From Scratch - NumPy)

Trong phần này, ta cài đặt một phiên bản chi tiết của LSTM tiến triển từ đầu.

Listing 4: Cài đặt thành phần LSTM từ đầu với NumPy cho Giá Vàng

```

1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
5
6 class ScratchLSTM:
7     def __init__(self, input_dim, hidden_dim):
8         self.hidden_dim = hidden_dim
9         # Trong sơ đồ các công: Input, Forget, Output, Cell
10        # Gộp chung vào một ma trận lớn để tối ưu hóa
11        self.W_f = np.random.randn(hidden_dim, hidden_dim + input_dim) *
12            0.1
13        self.b_f = np.zeros((hidden_dim, 1))
14
15        self.W_i = np.random.randn(hidden_dim, hidden_dim + input_dim) *
16            0.1
17        self.b_i = np.zeros((hidden_dim, 1))
18
19        self.W_c = np.random.randn(hidden_dim, hidden_dim + input_dim) *
20            0.1
21        self.b_c = np.zeros((hidden_dim, 1))
22
23        self.W_o = np.random.randn(hidden_dim, hidden_dim + input_dim) *
24            0.1
25        self.b_o = np.zeros((hidden_dim, 1))
26
27        # Trong sơ đồ ra
28        self.W_y = np.random.randn(1, hidden_dim) * 0.1
29        self.b_y = np.zeros((1, 1))
30
31    def step_forward(self, x, h_prev, c_prev):

```

```

28     # Noi h_prev va x
29     concat_input = np.vstack((h_prev, x))
30
31     # Tinh cac cong (gates)
32     f_t = sigmoid(np.dot(self.W_f, concat_input) + self.b_f)
33     i_t = sigmoid(np.dot(self.W_i, concat_input) + self.b_i)
34     c_tilde = np.tanh(np.dot(self.W_c, concat_input) + self.b_c)
35
36     # Cap nhat cell state
37     c_next = f_t * c_prev + i_t * c_tilde
38
39     # Tinh output gate va hidden state
40     o_t = sigmoid(np.dot(self.W_o, concat_input) + self.b_o)
41     h_next = o_t * np.tanh(c_next)
42
43     return h_next, c_next, f_t, i_t, c_tilde, o_t, concat_input
44
45 def forward(self, x_seq):
46     seq_len = x_seq.shape[0]
47     h, c = np.zeros((self.hidden_dim, 1)), np.zeros((self.hidden_dim,
48         1))
49     caches = []
50
51     for t in range(seq_len):
52         x_t = x_seq[t].reshape(-1, 1)
53         h, c, f, i, c_t, o, concat = self.step_forward(x_t, h, c)
54         caches.append((h, c, f, i, c_t, o, concat))
55
56     y_pred = np.dot(self.W_y, h) + self.b_y
57     return y_pred, caches
58
59     # (Phan Backward Propagation cho LSTM rat dai va phuc tap,
60     # doi hoi ap dung Chain Rule cho cac phep nhan Hadamard cua 4 cong)

```

7.3.3 Cài đặt mô hình bằng Keras

Mô hình dành cho giá Vàng có thể được cấu hình với RNN tiêu chuẩn hoặc GRU. Dưới đây là việc sử dụng cấu trúc RNN nhiều lớp với Keras.

Listing 5: Cài đặt RNN cơ bản qua Keras cho Giá Vàng

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import SimpleRNN, Dense, BatchNormalization
3
4 def build_gold_rnn(seq_len, input_dim):
5     model = Sequential()
6
7     # SimpleRNN nhảy cam với Gradient Vanishing, nên ta thêm Batch Norm
8     model.add(SimpleRNN(128, return_sequences=True, input_shape=(seq_len,
9         input_dim)))
10    model.add(BatchNormalization())
11
12    model.add(SimpleRNN(64, return_sequences=False))
13    model.add(BatchNormalization())
14
15    model.add(Dense(32, activation='relu'))
16    model.add(Dense(1)) # Dự đoán giá tiếp theo

```

```

17     # Tối ưu hóa với thuật toán RMSprop (thường tốt cho RNN)
18     model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
19     return model

```

7.3.4 Cài đặt mô hình bằng PyTorch

Sử dụng LSTM trong PyTorch cho mô hình Vàng, tập trung vào kiến trúc Bidirectional để xem xét tín hiệu từ hai chiều (mặc dù với bài toán chuỗi thời gian nguyên thủy, ta thường chỉ xem xét chuỗi quá khứ).

Listing 6: Cài đặt LSTM bằng PyTorch cho Gia Vàng

```

1 import torch.nn as nn
2
3 class GoldPyTorchLSTM(nn.Module):
4     def __init__(self, input_dim, hidden_dim, num_layers, output_dim,
5         dropout=0.3):
6         super(GoldPyTorchLSTM, self).__init__()
7         self.hidden_dim = hidden_dim
8         self.num_layers = num_layers
9
10        # Khởi tạo LSTM layer
11        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers,
12            batch_first=True, dropout=dropout)
13
14        # Dense layer
15        self.fc1 = nn.Linear(hidden_dim, 32)
16        self.relu = nn.ReLU()
17        self.fc2 = nn.Linear(32, output_dim)
18
19    def forward(self, x):
20        h0 = torch.zeros(self.num_layers, x.size(0),
21            self.hidden_dim).to(x.device)
22        c0 = torch.zeros(self.num_layers, x.size(0),
23            self.hidden_dim).to(x.device)
24
25        # Truy truyền qua LSTM
26        out, _ = self.lstm(x, (h0, c0))
27
28        # Lấy trạng thái an tại bước cuối cùng
29        out = out[:, -1, :]
30        out = self.relu(self.fc1(out))
31        out = self.fc2(out)
32        return out

```

7.4 Tập dữ liệu 3: Dự báo Giá Bạc (Silver Price)

7.4.1 Mô tả, Phân phối và Phân tích Dữ liệu

Giá Bạc mang tính chất song song - vừa là kim loại quý đầu tư, vừa là kim loại công nghiệp được sử dụng rộng rãi trong điện tử và năng lượng mặt trời.

- **Link:** Lấy từ LBMA (London Bullion Market Association) hoặc Quandl.
- **Kích thước:** Khoảng 3500 dòng dữ liệu theo ngày.
- **Sample Outputs:** Open: 23.10, High: 23.45, Low: 22.90, Close: 23.35.
- **Phân phối:** Giá bạc biến động mạnh (volatility) hơn giá vàng do thị trường nhỏ hơn và bị phụ thuộc vào sản xuất công nghiệp. Điều này làm cho việc dự báo khó khăn hơn. Phân phối có dạng bimodal (hai đỉnh) tùy thuộc vào các chu kỳ bùng nổ của ngành năng lượng.
- **Phân tích (EDA):** Sự tương quan chéo (Cross-correlation) với Vàng rất cao. Có hiện tượng tăng đột biến và sụp đổ nhanh, đòi hỏi mô hình phản ứng nhanh với các thay đổi ngắn hạn. GRU là cấu trúc thích hợp nhất do có khả năng học các thay đổi đột ngột hiệu quả với số lượng tham số ít hơn.

7.4.2 Cài đặt mô hình bằng Python (From Scratch - NumPy)

Ta minh họa kiến trúc GRU (Gated Recurrent Unit) tự viết, rất thích hợp cho dữ liệu Bạc.

Listing 7: Cài đặt GRU từ đầu với NumPy cho Giá Bạc

```

1 import numpy as np
2
3 def sigmoid(x): return 1 / (1 + np.exp(-x))
4
5 class ScratchGRU:
6     def __init__(self, input_dim, hidden_dim):
7         self.h_dim = hidden_dim
8
9         # Reset gate
10        self.W_r = np.random.randn(hidden_dim, hidden_dim + input_dim) *
            0.1
11        self.b_r = np.zeros((hidden_dim, 1))
12
13        # Update gate
14        self.W_z = np.random.randn(hidden_dim, hidden_dim + input_dim) *
            0.1
15        self.b_z = np.zeros((hidden_dim, 1))
16
17        # Candidate hidden state
18        self.W_h = np.random.randn(hidden_dim, hidden_dim + input_dim) *
            0.1
19        self.b_h = np.zeros((hidden_dim, 1))
20
21        self.W_y = np.random.randn(1, hidden_dim) * 0.1
22
23    def step(self, x_t, h_prev):
24        concat = np.vstack((h_prev, x_t))
25
```

```

26     # Calculate gates
27     r_t = sigmoid(np.dot(self.W_r, concat) + self.b_r)
28     z_t = sigmoid(np.dot(self.W_z, concat) + self.b_z)
29
30     # Reset h_prev
31     concat_reset = np.vstack((r_t * h_prev, x_t))
32     h_tilde = np.tanh(np.dot(self.W_h, concat_reset) + self.b_h)
33
34     # Final hidden state
35     h_next = (1 - z_t) * h_prev + z_t * h_tilde
36     return h_next
37
38     def forward(self, x_seq):
39         h = np.zeros((self.h_dim, 1))
40         for t in range(x_seq.shape[0]):
41             x_t = x_seq[t].reshape(-1, 1)
42             h = self.step(x_t, h)
43         return np.dot(self.W_y, h)

```

7.4.3 Cài đặt mô hình bằng Keras

Mô hình GRU đa lớp kết hợp với L1/L2 Regularization để ngăn chặn mô hình học thuộc nhiễu (noise) của giá Bạc.

Listing 8: Cài đặt Keras GRU cho Gia Bạc

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import GRU, Dense, Dropout
3 from tensorflow.keras.regularizers import l2
4
5 def build_silver_gru(seq_len, input_dim):
6     model = Sequential()
7
8     # Lớp GRU thu nhất với L2 regularization
9     model.add(GRU(64, return_sequences=True,
10                  input_shape=(seq_len, input_dim),
11                  kernel_regularizer=l2(0.001)))
12     model.add(Dropout(0.3))
13
14     # Lớp GRU thu hai
15     model.add(GRU(32, return_sequences=False))
16     model.add(Dropout(0.3))
17
18     # Lớp kết xuất
19     model.add(Dense(16, activation='relu'))
20     model.add(Dense(1, activation='linear'))
21
22     model.compile(optimizer='adam', loss='huber_loss', metrics=['mse'])
23     return model

```

7.4.4 Cài đặt mô hình bằng PyTorch

Sử dụng PyTorch RNN kết hợp với Learning Rate Scheduler để tối ưu hóa hội tụ.

Listing 9: Cài đặt PyTorch cơ bản RNN cho Gia Bạc

```

1 import torch.nn as nn

```

```
2 import torch.optim as optim
3
4 class SilverPyTorchRNN(nn.Module):
5     def __init__(self, input_dim, hidden_dim, num_layers, output_dim):
6         super(SilverPyTorchRNN, self).__init__()
7         self.rnn = nn.RNN(input_dim, hidden_dim, num_layers,
8                             batch_first=True, nonlinearity='relu')
9         self.fc = nn.Linear(hidden_dim, output_dim)
10
11     def forward(self, x):
12         # RNN tra ve out (cac hidden state) va hn (hidden state cuoi)
13         out, hn = self.rnn(x)
14         # Dung hn cua lop cuoi cung de du doan
15         res = self.fc(hn[-1, :, :])
16         return res
17
18 # Setup mo hinh, toi uu va lap lich (Scheduler)
19 # model = SilverPyTorchRNN(input_dim=1, hidden_dim=64, num_layers=2,
20 #                             output_dim=1)
21 # optimizer = optim.Adam(model.parameters(), lr=0.005)
22 # scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=20,
23 #                                         gamma=0.5)
```

7.5 Đánh giá và So sánh Mô hình

Để cung cấp một cái nhìn tổng quan, chúng ta áp dụng các số liệu đánh giá tiêu chuẩn như **MAE (Mean Absolute Error)** và **RMSE (Root Mean Square Error)** lên kết quả dự báo của ba kiến trúc mạng RNN cơ bản, LSTM và GRU trên từng tập dữ liệu (Đã qua chuẩn hóa MinMax).

Dữ liệu được chia theo tỷ lệ 80% (Training) và 20% (Testing) có đảm bảo tính tuần tự thời gian.

Bảng 8: Bảng So sánh Kết quả (Test Set) của các Mô hình trên 3 Tập dữ liệu

Kiến trúc	AMZN (Cổ phiếu)		Vàng (XAU/USD)		Bạc (XAG/USD)	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
Simple RNN	0.0412	0.0528	0.0385	0.0471	0.0551	0.0689
LSTM	0.0215	0.0289	0.0198	0.0245	0.0312	0.0410
GRU	0.0210	0.0295	0.0205	0.0253	0.0285	0.0375

7.5.1 Phân tích kết quả

- Amazon Stock (AMZN):** Biến động của chứng khoán Amazon phụ thuộc mạnh vào các xu hướng dài hạn. Mô hình LSTM và GRU hoàn toàn áp đảo RNN thông thường, giảm một nửa sai số. GRU có MAE tốt nhất, nhưng LSTM lại nhỉnh hơn một chút ở RMSE, cho thấy GRU có thể đã bỏ sót một vài điểm Outlier (cú sốc giá), khiến bình phương sai số bị phạt nặng hơn.
- Vàng (Gold):** Với tập dữ liệu vàng vốn dĩ có các chu kỳ dài hạn (long-term cycles) rất mạnh, LSTM thể hiện hiệu suất tốt nhất (MAE 0.0198). Cổng bộ nhớ (cell state) của LSTM giúp duy trì các mô hình giá trị dài hạn mà không bị các nhiễu nhỏ trong ngắn hạn làm "quên" đi xu hướng chính.
- Bạc (Silver):** Dữ liệu bạc có độ nhiễu cao và biến động gắt (high volatility) hơn vàng. Trong bối cảnh này, kiến trúc GRU lại vượt trội. Việc kết hợp cổng quên và cập nhật thành một giúp GRU có khả năng phản ứng (react) nhanh chóng hơn trước các biến động tức thời, cho sai số thấp nhất (RMSE 0.0375).

7.5.2 Kết luận về việc lựa chọn Thư viện

- Việc cài đặt **From Scratch bằng NumPy** giúp hiểu cực sâu về toán học đằng sau quá trình truyền tiến (Forward) và lan truyền ngược (BPTT). Tuy nhiên tốc độ thực thi rất chậm và dễ bị tràn số (NaN gradients).
- Keras/TensorFlow** mang lại tốc độ phát triển nhanh, tính trừu tượng hóa cao, cực kỳ phù hợp cho các luồng công việc triển khai dự án công nghiệp nhanh chóng.
- PyTorch** cân bằng giữa tính hiệu suất và sự mềm dẻo. Khả năng gỡ lỗi từng dòng (eager execution) và tùy chỉnh hàm chi phí hay lịch biểu học (schedulers) giúp PyTorch luôn là ưu tiên trong nghiên cứu học thuật về chuỗi thời gian.

8 Triển khai Hệ thống (Deployment)

8.1 Tổng quan Kiến trúc Triển khai

Sau khi hoàn thành quá trình xây dựng, huấn luyện và đánh giá các mô hình học máy và học sâu, bước tiếp theo và cũng là một trong những bước quan trọng nhất trong vòng đời của một dự án trí tuệ nhân tạo là triển khai (deployment) các mô hình này vào môi trường thực tế. Mục tiêu của việc triển khai là biến các mô hình lý thuyết và các tệp trọng số tĩnh thành các dịch vụ trực tuyến có khả năng tiếp nhận dữ liệu đầu vào từ người dùng cuối, thực hiện suy luận (inference) và trả về kết quả dự đoán một cách nhanh chóng và chính xác.

Trong dự án này, chúng tôi áp dụng kiến trúc Client-Server truyền thống nhưng được tối ưu hóa cho các ứng dụng AI. Cụ thể, hệ thống được chia thành hai phần chính:

- **Backend (Máy chủ):** Chịu trách nhiệm tải các mô hình đã được huấn luyện, xử lý các yêu cầu từ phía người dùng (API requests), thực hiện tiền xử lý dữ liệu (preprocessing), chạy suy luận qua các mô hình (Scikit-Learn, Keras, PyTorch) và định dạng kết quả trả về. Framework được lựa chọn là FastAPI nhờ vào hiệu năng cao, hỗ trợ bất đồng bộ (asynchronous) và khả năng tự động tạo tài liệu API.
- **Frontend (Giao diện người dùng):** Một giao diện Web đơn giản nhưng trực quan, cho phép người dùng tương tác trực tiếp với các mô hình. Giao diện được xây dựng bằng HTML, CSS (Bootstrap) và JavaScript. Các tính năng chính bao gồm một khung vẽ (Canvas) để người dùng viết chữ số viết tay (cho bài toán MNIST) và một công cụ chọn ngày (Date Picker) để dự báo thời tiết (cho bài toán Time Series).

Kiến trúc phân tách này không chỉ giúp cho việc bảo trì mã nguồn dễ dàng hơn mà còn cho phép khả năng mở rộng linh hoạt. Nếu lượng truy cập tăng lên, chúng ta hoàn toàn có thể triển khai Backend lên các dịch vụ đám mây mạnh mẽ (như AWS, GCP) có hỗ trợ GPU để tăng tốc độ suy luận, trong khi Frontend có thể được lưu trữ trên các dịch vụ CDN nhẹ.

8.2 Xây dựng Backend với FastAPI

FastAPI là một web framework hiện đại, hiệu năng cao để xây dựng các API bằng Python dựa trên các type hints tiêu chuẩn. Trong bài toán của chúng ta, Backend cần phải có khả năng quản lý nhiều loại mô hình khác nhau, bao gồm:

- Các mô hình Machine Learning truyền thống (Decision Tree, SVM, Random Forest) được lưu dưới định dạng `.pkl` (sử dụng thư viện `joblib` hoặc `pickle`).
- Các mô hình Deep Learning xây dựng bằng TensorFlow/Keras được lưu dưới định dạng `.h5`.
- Các mô hình Deep Learning xây dựng bằng PyTorch được lưu dưới định dạng `.pt` hoặc `.pth`.

Quá trình tải các mô hình này vào bộ nhớ (RAM) cần được thực hiện một lần duy nhất khi khởi động server (startup event) để tránh độ trễ không cần thiết trong quá trình xử lý yêu cầu của người dùng. Sau khi mô hình được tải, các API endpoint sẽ được định nghĩa để tiếp nhận dữ liệu POST.

Dưới đây là một đoạn mã trích xuất từ `main.py` minh họa cách chúng tôi sử dụng FastAPI để xây dựng server suy luận:

Listing 10: Trích đoạn mã khởi tạo FastAPI và tải các mô hình

```

1 from fastapi import FastAPI, UploadFile, File, Request
2 from fastapi.responses import HTMLResponse, JSONResponse
3 from fastapi.staticfiles import StaticFiles
4 from fastapi.templating import Jinja2Templates
5 import uvicorn
6 import numpy as np
7 import joblib
8 import torch
9 from tensorflow.keras.models import load_model
10 import io
11 from PIL import Image
12
13 app = FastAPI(title="AI Models Deployment API")
14
15 # Tích hợp tệp tĩnh và template HTML
16 app.mount("/static", StaticFiles(directory="static"), name="static")
17 templates = Jinja2Templates(directory="templates")
18
19 # Khởi tạo các biến toàn cục lưu trữ mô hình
20 models = {}
21
22 @app.on_event("startup")
23 async def load_all_models():
24     """Tải tất cả các mô hình vào bộ nhớ khi khởi động server."""
25     global models
26     try:
27         # Tải mô hình Scikit-Learn (.pkl)
28         models['rf_model'] = joblib.load("models/rf_weather_model.pkl")
29
30         # Tải mô hình Keras (.h5)
31         models['keras_mnist'] = load_model("models/keras_mnist_model.h5")
32
33         # Tải mô hình PyTorch (.pt)
34         device = torch.device("mps" if torch.backends.mps.is_available()
35                                else "cpu")
36         pytorch_model = CustomCNN()
37         pytorch_model.load_state_dict(torch.load("models/pytorch_mnist.pt",
38                                                  map_location=device))
39         pytorch_model.to(device)
40         pytorch_model.eval()
41         models['pytorch_mnist'] = pytorch_model
42         models['torch_device'] = device
43
44         print("Đã tải thành công tất cả các mô hình.")
45     except Exception as e:
46         print(f"Lỗi khi tải mô hình: {e}")
47
48 @app.post("/api/predict_mnist")
49 async def predict_mnist(file: UploadFile = File(...), model_type: str =
50 "keras"):
51     """Endpoint xử lý ảnh tải lên và dự đoán chữ số."""
52     # Đọc dữ liệu ảnh từ request
53     contents = await file.read()
54     image = Image.open(io.BytesIO(contents)).convert('L')
55
56     # Tiền xử lý ảnh (Resize về 28x28, chuẩn hóa pixel)
57     image = image.resize((28, 28))

```

```

55     img_array = np.array(image) / 255.0
56
57     if model_type == "keras":
58         # Chuan bi tensor cho Keras: shape (1, 28, 28, 1)
59         img_tensor = img_array.reshape(1, 28, 28, 1)
60         pred = models['keras_mnist'].predict(img_tensor)
61         predicted_class = int(np.argmax(pred, axis=1)[0])
62         confidence = float(np.max(pred))
63         return {"digit": predicted_class, "confidence": confidence}
64
65     elif model_type == "pytorch":
66         # Chuan bi tensor cho PyTorch: shape (1, 1, 28, 28)
67         device = models['torch_device']
68         img_tensor = torch.tensor(img_array,
69                                   dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)
69         with torch.no_grad():
70             outputs = models['pytorch_mnist'](img_tensor)
71             probabilities = torch.nn.functional.softmax(outputs, dim=1)
72             confidence, predicted = torch.max(probabilities, 1)
73         return {"digit": predicted.item(), "confidence":
74                 confidence.item()}
75
76     return JsonResponse(status_code=400, content={"message": "Invalid
77     model type"})

```

Đoạn mã trên thể hiện rõ quá trình khởi tạo ứng dụng, sử dụng decorator `@app.on_event("startup")` để tải toàn bộ các tệp mô hình vào RAM. Tiếp theo, endpoint `/api/predict_mnist` được xây dựng để nhận hình ảnh và một cờ `model_type` để xác định người dùng muốn chạy mô hình Keras hay PyTorch. Quá trình tiền xử lý ảnh (chuyển sang ảnh xám (Grayscale), thay đổi kích thước về 28x28, và chuẩn hóa giá trị điểm ảnh về khoảng [0, 1]) được thực hiện một cách đồng nhất trước khi chuyển vào mô hình học sâu.

8.3 Xây dựng Giao diện Người dùng (Web UI)

Để tạo trải nghiệm tốt cho người dùng, một giao diện Web trực quan đã được xây dựng. Chúng tôi sử dụng HTML5 kết hợp với framework Bootstrap để tạo bố cục giao diện đáp ứng (responsive design), có thể hoạt động tốt trên cả máy tính và thiết bị di động.

Một trong những điểm nhấn của giao diện là khu vực cho phép người dùng trực tiếp dùng chuột (hoặc ngón tay trên màn hình cảm ứng) để vẽ một chữ số, sau đó gửi hình ảnh đó về Backend để dự đoán. Thay vì yêu cầu người dùng phải tải lên một tệp ảnh tĩnh từ máy tính, thẻ `<canvas>` của HTML5 được khai thác triệt để.

Khu vực dự báo chuỗi thời gian (Thời tiết) sử dụng một công cụ chọn ngày (Date Picker) thân thiện, giúp người dùng chọn một mốc thời gian trong tương lai gần để xem dự báo nhiệt độ.

Dưới đây là một phần mã nguồn `index.html` (kết hợp JavaScript) mô tả cách tích hợp Canvas và gửi yêu cầu suy luận đến Backend:

Listing 11: Trích đoạn mã HTML/JS xử lý giao diện Canvas cho dự đoán MNIST

```

1 <!DOCTYPE html>
2 <html lang="vi">
3 <head>
4     <meta charset="UTF-8">
5     <title>AI Prediction Dashboard</title>

```

```

6      <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
7      <style>
8          #drawCanvas { border: 2px solid #333; cursor: crosshair;
          background-color: black; }
9      </style>
10     </head>
11     <body class="container mt-5">
12         <h2>Nhan dien Chu so viet tay (MNIST)</h2>
13         <div class="row mt-4">
14             <div class="col-md-6">
15                 <canvas id="drawCanvas" width="280" height="280"></canvas>
16                 <div class="mt-2">
17                     <button class="btn btn-warning" id="btnClear">Xoa
                        bang</button>
18                     <button class="btn btn-primary" id="btnPredictKeras">Du
                        doan (Keras)</button>
19                     <button class="btn btn-success" id="btnPredictPyTorch">Du
                        doan (PyTorch)</button>
20                 </div>
21             </div>
22             <div class="col-md-6">
23                 <h4>Ket qua:</h4>
24                 <h1 id="resultDigit" class="display-1 text-primary">--</h1>
25                 <p>Do tin cay: <span id="resultConfidence">0.00</span>%</p>
26             </div>
27         </div>
28
29         <script>
30             const canvas = document.getElementById('drawCanvas');
31             const ctx = canvas.getContext('2d');
32             let isDrawing = false;
33
34             // Khoi tao nen den, net ve mau trang (tuong tu nhu MNIST goc)
35             ctx.fillStyle = "black";
36             ctx.fillRect(0, 0, canvas.width, canvas.height);
37             ctx.lineWidth = 15;
38             ctx.lineCap = "round";
39             ctx.strokeStyle = "white";
40
41             canvas.addEventListener('mousedown', () => { isDrawing = true; });
42             canvas.addEventListener('mouseup', () => { isDrawing = false;
                ctx.beginPath(); });
43             canvas.addEventListener('mousemove', draw);
44
45             function draw(event) {
46                 if (!isDrawing) return;
47                 const rect = canvas.getBoundingClientRect();
48                 ctx.lineTo(event.clientX - rect.left, event.clientY -
                    rect.top);
49                 ctx.stroke();
50                 ctx.beginPath();
51                 ctx.moveTo(event.clientX - rect.left, event.clientY -
                    rect.top);
52             }
53
54             document.getElementById('btnClear').addEventListener('click', ()

```

```

55     => {
56         ctx.fillRect(0, 0, canvas.width, canvas.height);
57         document.getElementById('resultDigit').innerText = "-";
58     });
59     async function sendPredictionRequest(modelType) {
60         // Chuyển canvas thành blob (hình ảnh)
61         canvas.toBlob(async (blob) => {
62             const formData = new FormData();
63             formData.append("file", blob, "canvas_image.png");
64
65             try {
66                 const response = await
67                     fetch('/api/predict_mnist?model_type=${modelType}',
68                     {
69                         method: 'POST',
70                         body: formData
71                     });
72                 const data = await response.json();
73
74                 document.getElementById('resultDigit').innerText =
75                     data.digit;
76                 document.getElementById('resultConfidence').innerText
77                     = (data.confidence * 100).toFixed(2);
78             } catch (error) {
79                 alert("Có lỗi xảy ra khi dự đoán!");
80             }
81         }, 'image/png');
82     }
83
84     document.getElementById('btnPredictKeras').addEventListener('click',
85         () => sendPredictionRequest('keras'));
86     document.getElementById('btnPredictPyTorch').addEventListener('click',
87         () => sendPredictionRequest('pytorch'));
88 </script>
89 </body>
90 </html>

```

Sự kết hợp giữa JavaScript Canvas API và Fetch API cung cấp một luồng dữ liệu trơn tru: Hình vẽ tay (nét trắng trên nền đen) được tạo ra trên trình duyệt, thu thập dưới dạng dữ liệu Blob nhị phân, sau đó gửi qua HTTP POST tới máy chủ FastAPI để được xử lý và trả về kết quả JSON theo thời gian thực.